Examen Integral de Aplicaciones Distribuidas

Stack MERN

MongoDB · Express · React · Node.js

Evaluación teórica, práctica y proyecto integrador

Dato	Registro
Nombre del alumno	CARDOSO GUTIERREZ MARIO DIDIER
Boleta	2024640195
Fecha	10/11/2026
Calificación final (sobre 100)	100

Instrucciones generales

- El examen consta de tres partes: **Parte I** (fundamentos teóricos), **Parte II** (ejercicios prácticos por tecnología) y **Parte III** (proyecto integrador).
- Las Partes I y II se resuelven en una sesión presencial de **4 horas**. La Parte I se responde en este documento; las soluciones de la Parte II se entregan como archivos en un repositorio, con la ruta `parte2/P1.js`, `parte2/P2.js`, etc. (`.jsx` para React).
- La Parte III se desarrolla en un plazo de **2 semanas** y se cierra con una defensa oral de 20 minutos.
- En la Parte I, cada reactivo de opción múltiple tiene una sola respuesta correcta. Marque con una X o encierre el inciso.
- En la Parte II se permite consultar la documentación oficial (nodejs.org, expressjs.com, mongodb.com, mongoosejs.com, react.dev). No se permite el uso de asistentes de IA ni código de terceros.
- Versiones de referencia: Node.js 22 LTS o superior, Express 5, MongoDB 7 o superior, Mongoose 8 o superior, React 19, React Router 7 y Vite.

Parte	Contenido	Reactivos	Puntos
I	Fundamentos teóricos: opción múltiple (50 × 0.5), verdadero/falso (10 × 0.5) y preguntas abiertas (5 × 2)	65	40
II	Ejercicios prácticos: JavaScript, Node.js, Express, MongoDB, Mongoose, React e integración (10 × 3)	10	30
III	Proyecto integrador "HelpDesk MERN" con presentación oral	1	30
Total		76	100

> *Escala sugerida: 90–100 dominio sobresaliente; 80–89 dominio sólido; 70–79 dominio suficiente; menos de 70 no acredita. Para acreditar se requiere un mínimo de 60 % en cada parte.*

Parte I. Fundamentos teóricos (40 puntos)

Sección I.1 · Opción múltiple (50 reactivos × 0.5 = 25 puntos)

A. Arquitectura web y fundamentos del stack

1. En la arquitectura MERN, ¿qué componente es responsable de la persistencia de los datos?

a) React
b) Express
c) MongoDB
d) Node.js

Respuesta correcta: c) MongoDB

Explicación sencilla: MERN son las iniciales de cuatro piezas, cada una con un trabajo distinto: **M**ongoDB guarda los datos (base de datos), **E**xpress recibe peticiones y arma la API, **R**eact dibuja la pantalla en el navegador y **N**ode.js es el motor donde corre el servidor. Es como un restaurante: MongoDB es la bodega (guarda ingredientes), Express el mesero que lleva y trae pedidos, Node.js la cocina donde se trabaja y React el menú y el plato que ve el cliente.

Persistencia significa que los datos siguen existiendo aunque se apague el servidor; eso solo lo hace la base de datos (MongoDB). Por qué las otras no: React solo muestra la interfaz; Express enruta peticiones; Node.js ejecuta el código, pero lo que guarda en memoria se pierde al reiniciar.

2. ¿Cuál de los siguientes métodos HTTP es idempotente y se usa convencionalmente para reemplazar por completo un recurso?

a) POST
b) PUT
c) PATCH
d) OPTIONS

Respuesta correcta: b) PUT

Idempotente significa que repetir la misma petición 1, 2 o 100 veces deja el sistema en el mismo estado final. **PUT** reemplaza el recurso completo con lo que se envía: si lo mandas dos veces queda igual. Ejemplo: PUT /usuarios/5 con {nombre:"Ana", edad:30} deja el usuario 5 exactamente así, sin importar cuántas veces se ejecute.

Por qué las otras no: POST crea un recurso nuevo cada vez (no es idempotente: dos POST = dos registros); PATCH modifica solo algunos campos y no se garantiza que sea idempotente; OPTIONS solo consulta qué métodos acepta el servidor (se usa en CORS).

3. ¿Qué código de estado es el apropiado tras crear correctamente un recurso con POST?

a) 200 OK
b) 201 Created
c) 204 No Content
d) 302 Found

Respuesta correcta: b) 201 Created

Explicación sencilla: los códigos HTTP son “respuestas estándar” del servidor. La familia 2xx significa éxito, pero cada uno dice algo distinto: **200** = “todo bien, aquí tienes”; **201** = “se **creó** un recurso

nuevo" (lo correcto tras un POST de creación, y suele acompañarse de la cabecera Location); **204** = "éxito, pero no hay cuerpo que devolver" (típico de un DELETE); **302** es una redirección.

```
res.status(201).json(nuevoRecurso);
```

4. ¿Qué restricción de REST establece que el servidor no guarda contexto de sesión del cliente entre peticiones?

a) Cacheable

b) Sistema en capas

c) Sin estado (stateless)

d) Código bajo demanda

Respuesta correcta: c) Sin estado (stateless)

Explicación sencilla: REST define reglas para diseñar APIs. Una de ellas es que el servidor **no recuerda** nada de peticiones anteriores: cada petición debe llevar toda la información necesaria (por ejemplo, el token de sesión). Es como pedir en una ventanilla donde el empleado no te reconoce: cada vez muestras tu credencial. Esto facilita escalar, porque cualquier servidor de un grupo puede atender cualquier petición.

Por qué las otras no: *Cacheable* dice que las respuestas pueden guardarse en caché; *Sistema en capas* permite intermediarios (balanceadores, proxies); *Código bajo demanda* (opcional) permite enviar código ejecutable al cliente.

5. ¿Quién hace cumplir la política CORS?

a) Express

b) El navegador

c) MongoDB

d) El servidor DNS

Respuesta correcta: b) El navegador

Explicación sencilla: CORS (Cross-Origin Resource Sharing) es una regla de seguridad del **navegador**: una página cargada desde un origen (por ejemplo http://localhost:5173) no puede leer respuestas de otro origen (http://localhost:3000) a menos que ese servidor lo autorice con cabeceras como Access-Control-Allow-Origin. Express solo **envía** esas cabeceras (por ejemplo con el paquete cors); quien **bloquea** es el navegador. Por eso curl o Postman nunca son bloqueados por CORS (ver reactivo 59).

6. En una SPA construida con React y React Router, la navegación entre vistas:

a) Siempre solicita un HTML nuevo al servidor

b) Se resuelve en el cliente mediante la History API, sin recargar la página

c) Requiere WebSockets

d) Solo funciona con rutas hash (#)

Respuesta correcta: b) Se resuelve en el cliente mediante la History API, sin recargar la página

Explicación sencilla: una **SPA** (Single Page Application) carga un único HTML y después JavaScript "cambia" lo que se ve. React Router usa la **History API** del navegador (pushState) para cambiar la URL sin recargar y renderiza el componente que corresponde a la nueva ruta. Resultado: navegación instantánea y sin parpadeos; el servidor solo se consulta para obtener datos (JSON).

Por qué las otras no: a) eso hacen los sitios tradicionales; c) no usa WebSockets; d) también admite rutas normales (BrowserRouter); las rutas con # (HashRouter) son solo una opción.

7. ¿Cuál es la principal ventaja de MERN frente a stacks con lenguajes heterogéneos?

a) Es más rápido que cualquier backend en C++

b) Un solo lenguaje (JavaScript/TypeScript) y JSON como formato de datos de extremo a extremo

c) No requiere servidor

d) Elimina la necesidad de validar datos

Respuesta correcta: b) Un solo lenguaje (JavaScript/TypeScript) y JSON como formato de datos de extremo a extremo

Explicación sencilla: en MERN el navegador (React), el servidor (Node/Express) y hasta la base de datos (MongoDB almacena BSON, muy parecido a JSON) "hablan" el mismo idioma. Un objeto JavaScript viaja casi sin conversión desde la pantalla hasta la base y de regreso. Ventajas: un equipo puede trabajar en todo el stack, se reutilizan funciones y validaciones, y hay menos "traducción" de datos.

Por qué las otras no: a) no es cierto que sea más rápido que C++; c) siempre necesita servidor; d) la validación sigue siendo indispensable.

B. JavaScript moderno

8. ¿Qué imprime console.log(typeof null)?

a) "null"

b) "undefined"

c) "object"

d) "NaN"

Respuesta correcta: c) "object"

Explicación sencilla: typeof devuelve el tipo de un valor. Para null devuelve **"object"**; es un error histórico de JavaScript (desde 1995) que se conserva por compatibilidad. null representa "ausencia intencional de valor", pero no es realmente un objeto.

```
console.log(typeof null);       // "object"
console.log(typeof undefined);  // "undefined"
console.log(typeof NaN);        // "number"
```

Para comprobar null se usa **valor === null**, no typeof.

9. Dado el siguiente código, ¿cuál es el orden de salida?

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
```

a) A B C D

b) A D B C

c) A D C B

d) A C D B

Respuesta correcta: c) A D C B

Explicación sencilla: JavaScript ejecuta primero todo el código **síncrono** (línea por línea) y después atiende lo "pendiente", que se guarda en dos colas: las **microtareas** (promesas: .then, async/await) y las **macrotareas** (setTimeout, setInterval, E/S). Orden: primero todo lo síncrono → luego **todas** las microtareas → luego una macrotarea.

- 1.º console.log('A') → síncrono, imprime **A**.

- 2.º setTimeout(..., 0) → se manda a la cola de macrotareas (aunque el tiempo sea 0, espera su turno).
- 3.º Promise.resolve().then(...) → se manda a la cola de microtareas.
- 4.º console.log('D') → síncrono, imprime **D**.
- 5.º Se vacía la cola de microtareas → imprime **C**.
- 6.º Se ejecuta la macrotarea del setTimeout → imprime **B**.

Resultado: **A, D, C, B**.

10. ¿Cuál afirmación describe correctamente la diferencia entre let y var?

a) let tiene alcance de bloque y está sujeto a la zona muerta temporal (TDZ); var tiene alcance de función y se eleva inicializado como undefined

b) Son equivalentes

c) var tiene alcance de bloque y let de función

d) Una variable declarada con let no puede reasignarse

Respuesta correcta: a) let tiene alcance de bloque y está sujeto a la zona muerta temporal (TDZ); var tiene alcance de función y se eleva inicializado como undefined

Explicación sencilla: el **alcance** es la zona donde existe una variable. *var* vive en toda la **función**; *let* solo dentro del **bloque** { } donde se declaró. Además, con var el nombre se "eleva" (hoisting) al inicio y vale undefined antes de la línea de declaración; con let existe una **zona muerta temporal (TDZ)** y acceder antes de declararla lanza ReferenceError.

```
console.log(a);  // undefined (var se eleva)
var a = 1;


console.log(b);  // ReferenceError (TDZ)
let b = 2;


if (true) { var x = 1; let y = 2; }
console.log(x);  // 1
console.log(y);  // ReferenceError
```

Por qué las otras no: b) no son equivalentes; c) está invertida; d) let **sí** puede reasignarse (lo que no se puede reasignar es const).

11. Promise.all([p1, p2, p3]) se rechaza:

a) Solo si las tres promesas se rechazan

b) En cuanto cualquiera se rechaza, con la razón de la primera en rechazarse

c) Nunca; siempre devuelve los estados de cada promesa

d) Después de un tiempo límite predeterminado

Respuesta correcta: b) En cuanto cualquiera se rechaza, con la razón de la primera en rechazarse

Explicación sencilla: Promise.all recibe varias promesas y espera a que **todas** tengan éxito; entonces devuelve un arreglo con los resultados. Pero si **una sola** falla, Promise.all se rechaza de inmediato con el error de esa primera que falló ("fail-fast"), sin esperar a las demás.

```
await Promise.all([ok1, falla, ok2]);  // rechaza en cuanto falla
```

Si quieres esperar a todas sin importar fallos, se usa **Promise.allSettled** (devuelve el estado de cada una; ver ejercicio P1). **Por qué las otras no:** a) se rechaza con que falle una; c) eso es allSettled; d) no existe tiempo límite predeterminado.

12. Respecto a this, una función flecha (arrow function):

a) Crea su propio this en cada invocación

b) Hereda this léxicamente del ámbito que la contiene

c) Siempre tiene this igual a globalThis

d) Recibe como this el elemento del DOM que disparó el evento

Respuesta correcta: b) Hereda this léxicamente del ámbito que la contiene

Explicación sencilla: en una función normal, **this** depende de *cómo se llama* la función. Una **función flecha** no tiene su propio this: usa el this del lugar donde fue **escrita** (herencia léxica). Es muy útil en callbacks dentro de métodos o clases, para no perder el this.

```
const persona = {
  nombre: 'Ana',
  normal() { setTimeout(function () { console.log(this.nombre); }, 0); }, // undefined
  flecha() { setTimeout(() => console.log(this.nombre), 0); },           // "Ana"
};
```

13. ¿Qué imprime el siguiente código?

```
const a = { user: { name: 'Ana' }, tags: ['x'] };
const b = { ...a };
b.user.name = 'Luis';
console.log(a.user.name);
```

a) Ana

b) Luis

c) undefined

d) Lanza un error

Respuesta correcta: b) Luis

Explicación sencilla: el operador de propagación { ...a } hace una copia **superficial** (shallow copy): copia las propiedades del primer nivel, pero si una propiedad es un objeto (como user), solo copia la **referencia** (la "dirección de memoria"), no el objeto. Entonces a.user y b.user apuntan al **mismo** objeto; cambiar b.user.name también cambia a.user.name.

```
const b = { ...a };
b.user.name = 'Luis';
console.log(a.user.name);   // "Luis"

// Copia profunda (deep copy):
const c = structuredClone(a);
```

En React esto es importante: al actualizar estado hay que crear copias de cada nivel que cambie (inmutabilidad).

14. Una función declarada con async:

a) Devuelve el valor de return directamente

b) Siempre devuelve una Promise

c) Recibe un callback como último parámetro

d) Es un generador

Respuesta correcta: b) Siempre devuelve una Promise

Explicación sencilla: la palabra *async* convierte automáticamente el resultado de la función en una **Promise**. Si haces return 5, en realidad se devuelve Promise.resolve(5); si lanzas un error, se devuelve una promesa rechazada. Por eso se usa con await o .then().

```
async function f() { return 5; }
console.log(f());          // Promise { 5 }
f().then(v => console.log(v));  // 5
```

C. Node.js

15. Node.js ejecuta JavaScript con:

a) SpiderMonkey

b) El motor V8, apoyado en libuv para la E/S asíncrona y el event loop

c) La JVM

d) Chakra

Respuesta correcta: b) El motor V8, apoyado en libuv para la E/S asíncrona y el event loop

Explicación sencilla: Node.js no es un lenguaje, sino un **entorno de ejecución** para JavaScript fuera del navegador. Usa **V8** (el motor de Google Chrome) para entender y ejecutar el código, y **libuv** (una biblioteca en C) para manejar operaciones de entrada/salida (archivos, red) de forma asíncrona y el **event loop**.

Por qué las otras no: SpiderMonkey es el motor de Firefox; la JVM es de Java; Chakra era el motor del antiguo Edge.

16. ¿Cuál es la mejor estrategia para una tarea intensiva en CPU (por ejemplo, redimensionar imágenes) en un servidor Node sin bloquear las demás peticiones?

a) Envolverla en setTimeout

b) Declararla async y usar await

c) Ejecutarla en worker_threads (o en un proceso o servicio aparte, por ejemplo con una cola de trabajos)

d) Aumentar el tamaño del heap

Respuesta correcta: c) Ejecutarla en worker_threads (o en un proceso o servicio aparte, por ejemplo con una cola de trabajos)

Explicación sencilla: Node ejecuta tu JavaScript en **un solo hilo**. Si una tarea gasta mucho CPU (por ejemplo redimensionar imágenes), ese hilo se "ocupa" y **todas** las demás peticiones esperan. La solución es sacar el trabajo pesado del hilo principal: **worker_threads** (hilos de trabajo), procesos hijo (child_process / cluster), o un servicio/cola de trabajos (BullMQ, RabbitMQ) donde otro proceso lo realiza.

Por qué las otras no: envolver en setTimeout solo retrasa el trabajo, y sigue bloqueando cuando ejecuta; async/await no mueve el cálculo a otro hilo (solo ayuda con E/S); más heap sirve para memoria, no para CPU.

17. Para usar import/export nativos en archivos .js con Node.js:

a) No es posible

b) Declarar "type": "module" en package.json (o usar la extensión .mjs)

c) Es obligatorio instalar Babel

d) Usar require('import')

Respuesta correcta: b) Declarar "type": "module" en package.json (o usar la extensión .mjs)

Explicación sencilla: Node tiene dos sistemas de módulos: **CommonJS** (require / module.exports, el original) y **ES Modules** (import / export, el estándar). Por defecto un archivo .js se trata como CommonJS. Para usar import/export se agrega en package.json:

```
{
  "name": "mi-api",
  "type": "module"
}
```

o bien se nombra el archivo con extensión **.mjs**. No hace falta Babel; sí es posible, y no existe "require('import')".

18. ¿Cuál es el propósito de package-lock.json?

a) Listar los scripts del proyecto

b) Registrar las versiones exactas resueltas de todo el árbol de dependencias para tener instalaciones reproducibles

c) Guardar credenciales de npm

d) Configurar ESLint

Respuesta correcta: b) Registrar las versiones exactas resueltas de todo el árbol de dependencias para tener instalaciones reproducibles

Explicación sencilla: en package.json se escriben rangos de versión (por ejemplo "^4.18.0" = "4.18.0 o una más nueva compatible"). Como esos rangos cambian con el tiempo, dos personas podrían instalar versiones distintas. **package-lock.json** "congela" la versión exacta de cada paquete (y de las dependencias de las dependencias), así todos instalan lo mismo. Debe subirse a Git.

19. ¿Cuál es la diferencia entre dependencies y devDependencies?

a) Ninguna

b) devDependencies solo se necesitan para desarrollo, compilación o pruebas (nodemon, vitest, eslint) y pueden omitirse en producción

c) devDependencies se instalan de forma global

d) dependencies solo aplica al frontend

Respuesta correcta: b) devDependencies solo se necesitan para desarrollo, compilación o pruebas (nodemon, vitest, eslint) y pueden omitirse en producción

Explicación sencilla: *dependencies* son paquetes que la aplicación necesita para **funcionar** en producción (express, mongoose). *devDependencies* son herramientas para **construir y probar** (nodemon, eslint, vitest, supertest). En producción se instala solo lo necesario con **npm ci --omit=dev**, lo que reduce tamaño y superficie de ataque.

```
npm install express mongoose          # dependencies
npm install -D nodemon vitest supertest   # devDependencies
```

20. Los callbacks registrados con process.nextTick se ejecutan:

a) En la fase timers

b) Al terminar la operación actual, antes que las microtareas de las promesas

c) En la fase check

d) Después de setImmediate

Respuesta correcta: b) Al terminar la operación actual, antes que las microtareas de las promesas

Explicación sencilla: process.nextTick es una cola especial de Node con la **máxima prioridad**: sus callbacks se ejecutan en cuanto termina la operación en curso, **antes** de las microtareas de promesas y antes de seguir con las fases del event loop.

```
Promise.resolve().then(() => console.log('promesa'));
process.nextTick(() => console.log('nextTick'));
// Salida: nextTick, promesa
```

Orden de prioridad: código síncrono → nextTick → microtareas (promesas) → macrotareas (timers, setImmediate…). Usarlo en exceso puede "hambrear" al event loop.

21. ¿Cuál es la forma recomendada de manejar configuración sensible, como la cadena de conexión o el secreto del JWT?

a) Escribirla directamente en el código

b) Variables de entorno (process.env), con el archivo .env excluido del control de versiones y un .env.example documentado

c) Guardarla en el frontend

d) Dejarla en un comentario del README

Respuesta correcta: b) Variables de entorno (process.env), con el archivo .env excluido del control de versiones y un .env.example documentado

Explicación sencilla: los secretos (contraseñas de la base de datos, la clave para firmar JWT) **nunca** deben escribirse en el código ni subirse a Git, porque cualquiera con acceso al repositorio los vería. Se guardan en variables de entorno; en desarrollo se leen de un archivo **.env** (que se agrega a .gitignore) y se sube un **.env.example** con los nombres de las variables pero sin valores reales, para que otros sepan qué configurar.

```
# .env.example
PORT=3000
MONGODB_URI=mongodb://localhost:27017/helpdesk
JWT_SECRET=cambia_esto
```

D. Express

22. ¿Cuál es la firma de un middleware de manejo de errores en Express?

a) (req, res)

b) (req, res, next)

c) (err, req, res, next)

d) (err, res)

Respuesta correcta: c) (err, req, res, next)

Explicación sencilla: Express distingue un middleware "normal" de uno de **errores** por el **número de parámetros**: el de errores debe tener exactamente **cuatro** (err, req, res, next), aunque no uses next. Si

solo declaras tres, Express lo trata como middleware normal y nunca recibe los errores. Se registra **al final**, después de las rutas.

```
app.use((err, req, res, next) => {
  res.status(err.status ?? 500).json({ error: err.message });
});
```

23. Si no se registra app.use(express.json()), en una petición POST con cuerpo JSON el valor de req.body será:

a) El objeto ya parseado

b) undefined

c) La cadena JSON

d) Un Buffer

Respuesta correcta: b) undefined

Explicación sencilla: cuando llega un POST con JSON, el servidor recibe texto en bruto. El middleware **express.json()** lo lee y lo convierte en objeto dentro de **req.body**. Si no lo registras, en Express 5 req.body queda en **undefined** (en Express 4 era {}). Por eso casi toda API empieza con app.use(express.json()).

```
app.use(express.json());   // antes de las rutas
```

24. Para la URL /api/users/42?active=true y la ruta /api/users/:id:

a) req.query.id vale '42' y req.params.active vale 'true'

b) req.params.id vale '42' y req.query.active vale 'true'

c) req.body.id vale 42

d) req.params.id vale el número 42

Respuesta correcta: b) req.params.id vale '42' y req.query.active vale 'true'

Explicación sencilla: una URL puede llevar datos de dos formas. Los **parámetros de ruta** (:id) forman parte de la ruta y se leen en **req.params**; los **parámetros de consulta** (lo que va después del ?) se leen en **req.query**. Ambos llegan siempre como **cadenas de texto** (por eso '42' y 'true', no el número 42 ni el booleano true). El cuerpo (req.body) es para POST/PUT/PATCH.

```
// GET /api/users/42?active=true
req.params.id       // "42"
req.query.active    // "true"
Number(req.params.id)  // 42 (conversión manual)
```

25. Se registran en este orden app.get('/users/:id', h1) y después app.get('/users/stats', h2). Al solicitar GET /users/stats:

a) Responde h2

b) Responde h1, con req.params.id igual a 'stats'

c) Responde 404

d) Responden ambos

Respuesta correcta: b) Responde h1, con req.params.id igual a 'stats'

Explicación sencilla: Express evalúa las rutas **en el orden en que se registraron** y usa la **primera** que coincida. La ruta /users/:id coincide con *cualquier* texto después de /users/, incluido “stats”, y como

está registrada primero, h2 nunca se ejecuta. **Regla:** registrar primero las rutas específicas (/users/stats) y después las dinámicas (/users/:id).

```
app.get('/users/stats', h2);   // primero la específica
app.get('/users/:id', h1);      // después la dinámica
```

26. ¿Para qué sirve express.Router()?

a) Para crear un servidor HTTPS

b) Para agrupar rutas y middlewares en módulos montables bajo un prefijo

c) Para conectarse a MongoDB

d) Para renderizar componentes React

Respuesta correcta: b) Para agrupar rutas y middlewares en módulos montables bajo un prefijo

Explicación sencilla: express.Router() es una "mini-aplicación" para organizar el código por tema. En lugar de poner 50 rutas en un archivo, creas un router para usuarios, otro para tickets, etc. y los **montas** con un prefijo.

```
// routes/usuarios.js
const router = express.Router();
router.get('/', listar);
router.get('/:id', detalle);
export default router;

// app.js
app.use('/api/usuarios', usuariosRouter);  // /api/usuarios y /api/usuarios/:id
```

27. En Express 5, si un handler async lanza una excepción o devuelve una promesa rechazada:

a) El proceso de Node termina

b) El error se ignora

c) Express pasa el error a next(err) automáticamente y llega al manejador de errores

d) Express reintenta la petición

Respuesta correcta: c) Express pasa el error a next(err) automáticamente y llega al manejador de errores

Explicación sencilla: una función **async** devuelve una promesa; si dentro algo falla, esa promesa se **rechaza**. En Express 4 eso no se capturaba: había que usar try/catch con next(err) o paquetes especiales; si no, el error quedaba como rechazo no manejado. **Express 5** ya reconoce las promesas rechazadas y llama a next(err) por ti, así que el error llega a tu middleware de errores (reactivo 22).

```
app.get('/x', async (req, res) => {
  const datos = await Modelo.find();   // si lanza error → va a errorHandler
  res.json(datos);
});
```

28. ¿Qué ocurre si un handler ejecuta res.json(a) y luego res.json(b)?

a) Se envían ambos objetos

b) Se produce el error "Cannot set headers after they are sent to the client"

c) Los objetos se fusionan

d) Solo se envía b

Respuesta correcta: b) Se produce el error "Cannot set headers after they are sent to the client"

Explicación sencilla: una petición HTTP solo admite **una respuesta**. Cuando se ejecuta res.json(a), Express envía las cabeceras y el cuerpo. Si se intenta enviar una segunda respuesta, ya no se pueden modificar las cabeceras y Node lanza ese error (ERR_HTTP_HEADERS_SENT). Es un error típico cuando falta un **return** tras responder:

```
if (!usuario) {
  return res.status(404).json({ error: 'No encontrado' });  // con return
}
res.json(usuario);
```

E. MongoDB y Mongoose

29. ¿En qué formato almacena MongoDB los documentos?

a) JSON plano

b) BSON

c) XML

d) CSV

Respuesta correcta: b) BSON

Explicación sencilla: BSON significa “Binary JSON”. Es una versión **binaria** de JSON que MongoDB usa internamente. Se ve como JSON cuando consultas, pero por dentro se guarda en binario porque es más rápido de recorrer y soporta más tipos de datos que JSON (fechas, ObjectId, enteros de 32/64 bits, decimales, datos binarios).

30. ¿Cuál es el tamaño máximo de un documento en MongoDB?

a) 1 MB

b) 16 MB

c) 64 MB

d) No tiene límite

Respuesta correcta: b) 16 MB

Explicación sencilla: un solo documento de MongoDB no puede pesar más de **16 megabytes**. Este límite evita que un documento consuma demasiada memoria y obliga a un buen diseño: por ejemplo, un arreglo que crece sin límite (comentarios de una publicación muy popular) **no** debe embeberse, sino guardarse en otra colección. Para archivos grandes se usa GridFS.

31. ¿Qué operador de actualización incrementa de forma atómica un campo numérico?

a) $set

b) $inc

c) $push

d) $add

Respuesta correcta: b) $inc

Explicación sencilla: $inc suma (o resta, con número negativo) a un campo numérico **de forma atómica**: la operación completa la hace la base de datos de una sola vez, sin que otra petición se “cuele” en medio. Evita el clásico problema de leer, sumar en JavaScript y guardar, donde dos usuarios simultáneos podrían pisarse.

```
db.productos.updateOne({ _id: id }, { $inc: { stock: -1 } });
```

Por qué las otras no: $set asigna un valor; $push agrega a un arreglo; $add es un operador de agregación, no de actualización.

32. ¿Cuándo es preferible embeber documentos en lugar de referenciarlos?

a) En relaciones uno-a-muchos sin límite de crecimiento

b) Cuando los datos se leen juntos, la relación es uno-a-pocos y el crecimiento está acotado

c) Siempre

d) Cuando los subdocumentos se consultan de forma independiente con frecuencia

Respuesta correcta: b) Cuando los datos se leen juntos, la relación es uno-a-pocos y el crecimiento está acotado

Explicación sencilla: en MongoDB hay dos formas de relacionar datos. **Embeber** = meter el dato dentro del documento (como guardar las líneas de una factura dentro de la factura); **referenciar** = guardar solo el _id y el dato vive en otra colección. Se embebe cuando siempre se leen juntos, son pocos y no crecen sin control. Se referencia cuando son muchos, crecen sin límite, o se consultan por separado.

Por qué las otras no: a) y d) son casos para **referenciar**; c) "siempre" nunca es buen criterio de diseño.

33. ¿Qué etapa de agregación realiza el equivalente a un join con otra colección?

a) $group

b) $lookup

c) $unwind

d) $project

Respuesta correcta: b) $lookup

Explicación sencilla: un **join** une datos de dos tablas/colecciones usando un campo común. En el pipeline de agregación de MongoDB lo hace **$lookup**:

```
db.orders.aggregate([
  { $lookup: {
      from: 'customers',          // colección a unir
      localField: 'customerId',   // campo en orders
      foreignField: '_id',        // campo en customers
      as: 'cliente'               // nombre del arreglo resultado
  } }
]);
```

Por qué las otras no: $group agrupa y calcula totales; $unwind "desenrolla" un arreglo en varios documentos; $project elige o crea campos.

34. Para la consulta find({ status: 'paid', total: { $gt: 1000 } }).sort({ createdAt: -1 }), ¿qué índice compuesto sigue la regla ESR?

a) { createdAt: -1, total: 1, status: 1 }

b) { status: 1, createdAt: -1, total: 1 }

c) { total: 1, status: 1, createdAt: -1 }

d) El orden de los campos no importa

Respuesta correcta: **b) { status: 1, createdAt: -1, total: 1 }**

Explicación sencilla: un **índice compuesto** es como un directorio ordenado por varios campos a la vez; el **orden de los campos** importa. La regla **ESR** dice que se ordenen así: primero los campos con **E**quality (igualdad), luego los de **S**ort (ordenamiento) y al final los de **R**ange (rango).

- Igualdad: status = "paid" → primero.
- Ordenamiento: sort por createdAt descendente → segundo.
- Rango: total > 1000 → al final.

```
db.orders.createIndex({ status: 1, createdAt: -1, total: 1 })
```

Así MongoDB va directo a los "paid", ya los encuentra ordenados por fecha (sin hacer un SORT en memoria) y filtra el rango dentro del índice.

35. Las transacciones ACID multi-documento en MongoDB requieren:

a) Un servidor standalone

b) Un replica set o un clúster fragmentado (sharded)

c) Usar Mongoose obligatoriamente

d) No existen en MongoDB

Respuesta correcta: **b) Un replica set o un clúster fragmentado (sharded)**

Explicación sencilla: una **transacción ACID** agrupa varias operaciones para que o se hagan **todas** o **ninguna** (por ejemplo, restar saldo a una cuenta y sumarlo a otra). En MongoDB las transacciones multi-documento requieren un **replica set** (varios servidores con copias de los datos) o un clúster con sharding. Un servidor standalone no las admite (en desarrollo se puede convertir en un replica set de un solo nodo). No dependen de Mongoose; el driver oficial también las soporta.

36. En Mongoose, populate('author'):

a) Crea índices en la colección

b) Sustituye el ObjectId referenciado por el documento de la colección referida, ejecutando una consulta adicional

c) Ejecuta un join dentro del servidor de MongoDB en una sola operación

d) Valida el esquema del documento

Respuesta correcta: **b) Sustituye el ObjectId referenciado por el documento de la colección referida, ejecutando una consulta adicional**

Explicación sencilla: cuando un documento solo guarda el _id de otro (por ejemplo, un ticket guarda el id del autor), populate() "rellena" ese campo con el documento completo. Lo hace Mongoose **en el servidor Node** lanzando una **segunda consulta** (no es un join dentro de MongoDB; eso sería $lookup).

```
const ticket = await Ticket.findById(id).populate('author', 'nombre email');
// ticket.author ahora es { _id, nombre, email } en lugar de solo el ObjectId
```

F. React

37. ¿Cuál es el propósito de la prop key en las listas?

a) Aplicar estilos

b) Identificar de forma estable cada elemento para que la reconciliación conserve correctamente el estado y el DOM

c) Proteger contra XSS

d) Poder leer props.key dentro del componente

Respuesta correcta: b) Identificar de forma estable cada elemento para que la reconciliación conserve correctamente el estado y el DOM

Explicación sencilla: cuando React dibuja una lista, necesita saber cuál elemento es cuál entre un render y el siguiente, para actualizar solo lo que cambió. La prop **key** es la "etiqueta de identidad" de cada elemento. Debe ser **única y estable** (por ejemplo el _id de la base de datos), no el índice del arreglo si la lista puede reordenarse o borrarse, porque se mezclarían estados (por ejemplo lo escrito en un input).

```
{tickets.map(t => <li key={t._id}>{t.titulo}</li>)}
```

Por qué las otras no: key no sirve para estilos ni seguridad, y React **no** la pasa al componente (props.key es undefined).

38. useEffect(() => { ... }, []) se ejecuta:

a) En cada render

b) Después del primer montaje (en desarrollo, StrictMode monta, desmonta y vuelve a montar para detectar efectos sin limpieza)

c) Antes del render

d) Nunca

Respuesta correcta: b) Después del primer montaje (en desarrollo, StrictMode monta, desmonta y vuelve a montar para detectar efectos sin limpieza)

Explicación sencilla: useEffect ejecuta código "secundario" (pedir datos, suscribirse, etc.) **después** de que React pinta el componente. El segundo argumento es el arreglo de **dependencias**: sin arreglo = en cada render; con dependencias = cuando cambian; con **[]** (vacío) = solo al montar (la primera vez). En desarrollo, <StrictMode> monta, desmonta y vuelve a montar a propósito para que descubras efectos que no limpian lo que crean; por eso verás dos ejecuciones, pero en producción es una sola.

39. ¿Cuál es la forma correcta de agregar un elemento x a un estado de tipo arreglo llamado items?

a) items.push(x); setItems(items);

b) setItems(prev => [...prev, x]);

c) items[items.length] = x;

d) setItems(items.concat);

Respuesta correcta: b) setItems(prev => [...prev, x]);

Explicación sencilla: en React el estado es **inmutable**: nunca se modifica el arreglo existente; se crea **uno nuevo** con el cambio y se pasa al setter. React compara referencias; si recibe el mismo arreglo, cree que nada cambió y no vuelve a renderizar. La forma con función (prev => ...) garantiza usar el valor más reciente del estado.

```
// Incorrecto: muta el arreglo original
items.push(x); setItems(items);

// Correcto: crea un arreglo nuevo
setItems(prev => [...prev, x]);
```

40. ¿Cuál es la solución nativa de React para evitar el prop drilling de datos globales como el usuario autenticado?

a) Redux, que es obligatorio

b) Context API (createContext y useContext)

c) localStorage

d) Refs

Respuesta correcta: b) Context API (createContext y useContext)

Explicación sencilla: el *prop drilling* ocurre cuando un dato (el usuario autenticado) debe pasar por muchos componentes intermedios que ni lo usan, solo para llegar a uno profundo. El **Context** crea un "canal" global: un componente **Provider** coloca el dato y cualquier componente hijo lo lee con useContext sin recibirlo por props.

```
const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  return <AuthContext.Provider value={{ user, setUser }}>{children}</AuthContext.Provider>;
}

const { user } = useContext(AuthContext);   // en cualquier componente hijo
```

Redux es una alternativa, pero **no es obligatorio**; localStorage no actualiza la interfaz; las refs no provocan re-render.

41. ¿Cuál es la diferencia entre useMemo y useCallback?

a) Son idénticos

b) useMemo memoriza un valor calculado; useCallback memoriza la referencia de una función

c) useCallback memoriza valores y useMemo funciones

d) useMemo sirve para efectos secundarios

Respuesta correcta: b) useMemo memoriza un valor calculado; useCallback memoriza la referencia de una función

Explicación sencilla: en cada render, React vuelve a ejecutar todo el cuerpo del componente: recalcula valores y crea funciones nuevas. Memorizar evita trabajo repetido: **useMemo** guarda el **resultado** de un cálculo costoso mientras sus dependencias no cambien; **useCallback** guarda la **función misma** (misma referencia) para no romper la optimización de componentes hijos envueltos en React.memo.

```
const total = useMemo(() => items.reduce((s, i) => s + i.precio, 0), [items]);
const borrar = useCallback((id) => setItems(p => p.filter(i => i.id !== id)), []);
```

42. Un componente de formulario controlado es aquel en que:

a) El valor del input lo gestiona el DOM y se lee con una ref

b) El valor del input proviene del estado de React y se actualiza mediante onChange

c) No existe ningún estado

d) Los campos son de solo lectura

Respuesta correcta: **b) El valor del input proviene del estado de React y se actualiza mediante onChange**

Explicación sencilla: en un input **controlado**, React es "el dueño" del valor: lo que se ve en el campo siempre viene del estado, y cada tecla actualiza el estado con onChange. Permite validar, formatear o limpiar mientras se escribe. En uno **no controlado** el DOM guarda el valor y se lee con una ref.

```
const [nombre, setNombre] = useState('');
<input value={nombre} onChange={(e) => setNombre(e.target.value)} />
```

43. ¿Cuál de las siguientes prácticas viola las Reglas de los Hooks?

a) Llamar useState al inicio del componente

b) Llamar useEffect dentro de un if o de un bucle

c) Un custom hook que llama a useState

d) Llamar useContext dentro de un componente

Respuesta correcta: **b) Llamar useEffect dentro de un if o de un bucle**

Explicación sencilla: React identifica cada hook por el **orden** en que se llaman en cada render. Si un hook está dentro de un if o un bucle, a veces se llama y a veces no, el orden cambia y React se confunde ("Rendered more hooks than during the previous render"). **Reglas:** (1) llamar hooks solo en el **nivel superior** del componente (no dentro de condiciones, bucles ni funciones anidadas) y (2) solo desde componentes o **custom hooks**.

```
// Incorrecto
if (activo) { useEffect(() => {...}); }

// Correcto: la condición va dentro del hook
useEffect(() => { if (activo) {...} }, [activo]);
```

44. En React Router (v6 o superior), ¿cómo se lee el parámetro :id de la ruta /tickets/:id?

a) useLocation().id

b) useParams()

c) props.match.params

d) window.id

Respuesta correcta: **b) useParams()**

Explicación sencilla: en React Router, las partes dinámicas de la ruta (:id) se leen con el hook **useParams()**, que devuelve un objeto con esos valores (siempre cadenas de texto).

```
// <Route path="/tickets/:id" element={<DetalleTicket />} />
function DetalleTicket() {
  const { id } = useParams();   // /tickets/123 → id = "123"
}
```

Por qué las otras no: useLocation da la URL actual (pathname, search); props.match.params es de React Router v5 y se eliminó; window.id no existe.

G. Integración, seguridad, calidad y despliegue

45. ¿Qué forma de almacenar el JWT en el navegador mitiga su robo mediante XSS?

a) localStorage

b) Cookie con los atributos HttpOnly, Secure y SameSite

c) Una variable global de JavaScript

d) La cadena de consulta de la URL

Respuesta correcta: b) Cookie con los atributos HttpOnly, Secure y SameSite

Explicación sencilla: un ataque **XSS** inyecta JavaScript malicioso en tu página. Si el token está en localStorage, ese script puede leerlo y robarlo. Una cookie con **HttpOnly** no es visible para JavaScript, por lo que XSS no puede leerla. **Secure** hace que solo viaje por HTTPS y **SameSite** (Lax o Strict) reduce el riesgo de CSRF (peticiones falsificadas desde otros sitios).

```
res.cookie('token', jwt, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'lax',
  maxAge: 15 * 60 * 1000,
});
```

46. ¿Cuál es la forma correcta de almacenar contraseñas?

a) Cifradas con AES para poder recuperarlas

b) Con un hash lento y con sal, como bcrypt o argon2

c) Con MD5

d) Codificadas en Base64

Respuesta correcta: b) Con un hash lento y con sal, como bcrypt o argon2

Explicación sencilla: una contraseña **nunca** se guarda en claro ni cifrada de forma reversible: si alguien roba la base de datos, no debe poder recuperarla. Se guarda un **hash** (resultado de una función de un solo sentido). Debe ser **lento a propósito** (para que probar millones de combinaciones sea carísimo) y llevar una **sal** (valor aleatorio distinto por usuario, para que dos contraseñas iguales produzcan hashes distintos). bcrypt y argon2 hacen ambas cosas.

Por qué las otras no: AES es reversible (con la clave se recupera); MD5 es rapidísimo y está roto; Base64 es solo una codificación, no protege nada.

47. Considere User.findOne({ email: req.body.email, password: req.body.password }). Si un atacante envía {"email": "admin@x.com", "password": {"$ne": null}}, ¿qué sucede?

a) La consulta falla siempre

b) Inyección NoSQL: la condición se cumple sin conocer la contraseña

c) MongoDB lanza un error de tipo

d) Express bloquea el operador automáticamente

Respuesta correcta: b) Inyección NoSQL: la condición se cumple sin conocer la contraseña

Explicación sencilla: en MongoDB las consultas son objetos. Si el servidor pone directamente lo que envía el usuario, un atacante puede mandar un **operador** en lugar de un texto. Con {"password": {"$ne": null}} la consulta pregunta “usuario con ese email y contraseña **distinta de null**”, que es verdadero para cualquier usuario con contraseña: entra como admin sin conocerla.

```
// Consulta resultante:
```

```
User.findOne({ email: 'admin@x.com', password: { $ne: null } })

// Defensas:
// 1) validar que email y password sean cadenas (zod / joi)
// 2) express-mongo-sanitize o String(req.body.password)
// 3) comparar el hash con bcrypt (nunca filtrar por password en la consulta)
```

48. En un proyecto React con Vite, la variable VITE_API_URL definida en .env:

a) Es secreta

b) Se incrusta en el bundle del cliente y es pública; nunca debe contener secretos

c) Solo existe en el servidor

d) Se cifra automáticamente

Respuesta correcta: b) Se incrusta en el bundle del cliente y es pública; nunca debe contener secretos

Explicación sencilla: Vite toma las variables que empiezan con **VITE_** y las **escribe dentro del JavaScript** que se descarga al navegador. Cualquiera puede abrir las herramientas de desarrollador y verlas. Por eso sirven para datos **públicos** (la URL de la API) y **jamás** para claves secretas (JWT_SECRET, contraseñas de bases de datos); esos van solo en el servidor.

49. ¿Qué herramienta permite pruebas de integración de endpoints Express sin abrir un puerto real?

a) Supertest (con Vitest, Jest o node:test)

b) Únicamente Cypress

c) MongoDB Compass

d) Webpack

Respuesta correcta: a) Supertest (con Vitest, Jest o node:test)

Explicación sencilla: Supertest permite "llamar" a tu aplicación Express desde el código de la prueba (supertest(app).get('/api/x')) sin abrir un puerto real, y revisar código de estado y respuesta. Se usa junto a un ejecutor de pruebas (Vitest, Jest o el nativo node:test).

```
import request from 'supertest';
import app from '../src/app.js';

it('responde 401 sin sesión', async () => {
  const res = await request(app).get('/api/tickets');
  expect(res.status).toBe(401);
});
```

Por qué las otras no: Cypress hace pruebas de extremo a extremo con navegador; Compass es una interfaz gráfica de MongoDB; Webpack empaqueta código.

50. ¿Qué middleware de Express establece cabeceras HTTP de seguridad (Content-Security-Policy, X-Content-Type-Options, etc.)?

a) morgan

b) helmet

c) cors

d) body-parser

Respuesta correcta: b) helmet

Explicación sencilla: helmet es un middleware que agrega de una vez varias cabeceras HTTP de seguridad (Content-Security-Policy, X-Content-Type-Options, Strict-Transport-Security, etc.) que protegen contra ataques comunes como XSS o clickjacking.

```
import helmet from 'helmet';
app.use(helmet());
```

Por qué las otras no: morgan registra las peticiones (logs); cors habilita orígenes cruzados; body-parser interpreta el cuerpo de las peticiones.

Sección I.2 · Verdadero o falso (10 reactivos × 0.5 = 5 puntos)

Escriba V (verdadero) o F (falso) dentro del paréntesis.

51. ( ) MongoDB exige definir un esquema fijo antes de insertar documentos en una colección.

Respuesta correcta: FALSO

Justificación: MongoDB es **flexible en cuanto al esquema**: en una colección puedes insertar documentos con campos distintos sin declarar nada antes. (Se puede **opcionalmente** exigir validación con $jsonSchema, y Mongoose define esquemas en la capa de aplicación, pero MongoDB en sí no lo exige).

52. ( ) Node.js ejecuta el JavaScript del usuario en un único hilo principal.

Respuesta correcta: VERDADERO

Justificación: Node ejecuta **tu código JavaScript** en un solo hilo (el hilo principal), por eso una tarea pesada de CPU lo bloquea. Internamente libuv usa un pequeño grupo de hilos para ciertas operaciones de E/S (archivos, DNS, criptografía), pero tu código no corre en paralelo.

53. ( ) app.use('/api', router) monta todas las rutas del router bajo el prefijo /api.

Respuesta correcta: VERDADERO

Justificación: app.use('/api', router) **monta** el router con ese prefijo: una ruta router.get('/usuarios') queda disponible como GET /api/usuarios.

54. ( ) En React, mutar directamente un arreglo del estado y llamar al setter con la misma referencia provoca un nuevo render.

Respuesta correcta: FALSO

Justificación: React compara la referencia del nuevo estado con la anterior; si es **el mismo arreglo** (misma referencia) concluye que no cambió y **no vuelve a renderizar** (aunque el contenido haya sido mutado). Hay que crear un arreglo nuevo: setItems(prev => [...prev, x]).

55. ( ) Un JWT firmado está cifrado, por lo que su payload no puede leerse sin la clave.

Respuesta correcta: FALSO

Justificación: Un JWT firmado **no está cifrado**: tiene tres partes (cabecera.payload.firma) codificadas en Base64url, que **cualquiera puede decodificar y leer**. La firma solo garantiza que **no fue alterado**. Por eso no se deben guardar datos sensibles (contraseñas) en el payload. (Para cifrarlo existe JWE, que es otro estándar).

56. ( ) En Mongoose, findOneAndUpdate ejecuta por defecto los validadores del esquema.

Respuesta correcta: FALSO

Justificación: findOneAndUpdate (y updateOne, findByIdAndUpdate…) **no ejecuta los validadores por defecto**. Se activan con la opción { runValidators: true }; además, las validaciones del esquema con "this" solo funcionan en save().

57. ( ) Los hooks de React pueden invocarse dentro de una condición if siempre que la condición sea estable.

Respuesta correcta: FALSO

Justificación: Las Reglas de los Hooks prohíben llamarlos dentro de condiciones, **sin excepción**, aunque la condición “parezca estable”: React depende del orden de las llamadas en cada render. La condición debe ir **dentro** del hook (por ejemplo, dentro del efecto).

58. ( ) Un índice único sobre el campo email evita duplicados incluso cuando ocurren inserciones concurrentes.

Respuesta correcta: VERDADERO

Justificación: El índice único lo **hace cumplir MongoDB** de forma atómica: si dos inserciones simultáneas traen el mismo email, una de ellas falla con el error E11000 (duplicate key). Verificar en código con findOne y luego insertar **no** es seguro, porque entre las dos operaciones puede colarse otra petición.

59. ( ) La política CORS protege a la API contra peticiones hechas con curl o Postman.

Respuesta correcta: FALSO

Justificación: CORS lo aplican los **navegadores**; curl, Postman o un script de Node **no lo respetan**. CORS no es un mecanismo de autenticación ni protege a la API de clientes que no son navegadores; para eso hacen falta autenticación, autorización y rate limiting.

60. ( ) npm ci instala exactamente lo indicado en package-lock.json y falla si este no coincide con package.json.

Respuesta correcta: VERDADERO

Justificación: npm ci ("clean install") instala **exactamente** las versiones de package-lock.json (borra node_modules primero) y **falla** si package.json y el lock no coinciden. Por eso se usa en integración continua y despliegues; npm install, en cambio, puede actualizar el lock.

Sección I.3 · Preguntas abiertas (5 reactivos × 2 = 10 puntos)

Responda con claridad y precisión; se valora el uso correcto de la terminología y los ejemplos concretos.

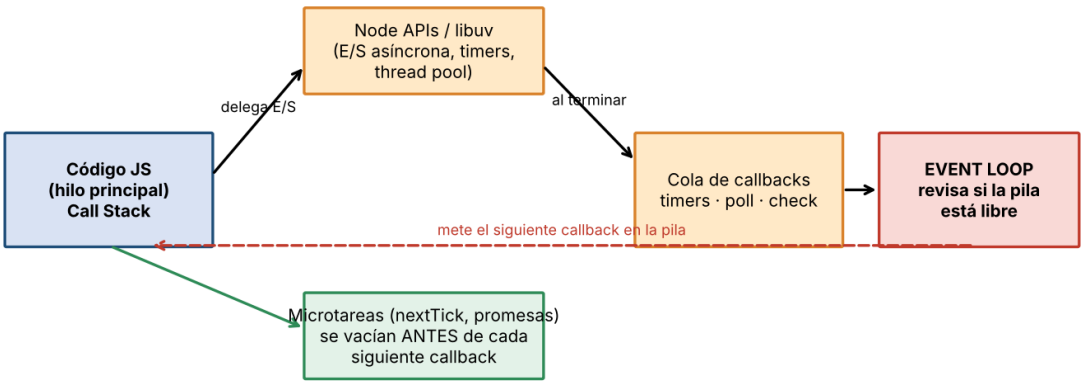
61. Explique el funcionamiento del event loop de Node.js. ¿Por qué Node es adecuado para aplicaciones con mucha E/S y poco adecuado para cómputo intensivo? Proponga dos soluciones para este último caso.

Respuesta 61

Idea previa. JavaScript solo puede hacer **una cosa a la vez** (un hilo). Aun así, un servidor Node atiende miles de conexiones porque no se queda "esperando" las operaciones lentas (leer un archivo, consultar la base de datos, llamar a otra API): las **delega** al sistema operativo y sigue atendiendo a otros. Quien coordina ese reparto es el **event loop** (bucle de eventos).

1) Funcionamiento del event loop

Event loop de Node.js (esquema simplificado)



- **Call stack (pila de llamadas):** donde se ejecuta el código JavaScript, una función a la vez.
- **libuv / APIs de Node:** cuando el código pide una operación asíncrona (fs.readFile, una consulta a MongoDB, setTimeout), Node se la entrega a libuv/al sistema operativo y **sigue ejecutando** el resto.
- **Colas de callbacks:** al terminar la operación, su *callback* (o la resolución de la promesa) se coloca en una cola.
- **Event loop:** repite sin parar: "¿la pila está vacía? → entonces toma el siguiente callback de la cola y ponlo a ejecutar". Sus fases principales son: *timers* (setTimeout/setInterval), *pending callbacks*, *poll* (nueva E/S), *check* (setImmediate) y *close*.
- **Prioridades:** entre fases se vacían primero las colas de **process.nextTick** y luego las **microtareas** (promesas).

2) Por qué es bueno para E/S y malo para CPU

- **Mucha E/S, poco cálculo (ideal):** mientras la base de datos responde, el hilo está libre para atender otras peticiones; con un solo hilo se manejan miles de conexiones concurrentes con poca memoria (APIs REST, chats, streaming).
- **Cómputo intensivo (malo):** si una función hace un cálculo largo (procesar una imagen, ordenar millones de datos, un bucle enorme), el hilo único queda **ocupado** y el event loop no puede avanzar: **todas** las peticiones esperan (la aplicación "se congela").

3) Dos soluciones para tareas de CPU

- **worker_threads:** crear hilos de trabajo dentro del mismo proceso para el cálculo pesado; se comunican con el hilo principal mediante mensajes.

```
import { Worker } from 'node:worker_threads';
const w = new Worker('./redimensionar.js', { workerData: { ruta } });
w.on('message', (resultado) => res.json(resultado));
```

- **Cola de trabajos + servicio aparte** (BullMQ/Redis, RabbitMQ): la API solo registra el trabajo y responde (por ejemplo 202 Accepted); otro proceso o servidor lo ejecuta. Alternativas: **cluster / child_process** para repartir en varios procesos y delegar el cálculo a un servicio escrito en un lenguaje más adecuado.

---

62. Para un blog con autores, publicaciones y comentarios, compare el modelado embebido y el referenciado en MongoDB. Proponga un diseño, justifíquelo e indique sus ventajas y desventajas.

Respuesta 62

Idea previa. En MongoDB hay dos maneras de relacionar datos: **embeber** (guardar el dato anidado dentro del documento) y **referenciar** (guardar solo el _id del otro documento, que vive en otra colección, parecido a una llave foránea).

Aspecto	Embebido	Referenciado
Lecturas	Una sola consulta trae todo (rápido)	Requieren populate/$lookup (consultas extra)
Escrituras	Un documento: operación atómica	Varias colecciones: coordinar o usar transacciones
Crecimiento	Limitado (documento máx. 16 MB)	Sin límite práctico
Consulta independiente	Difícil (hay que “entrar” al padre)	Fácil (colección propia)
Duplicación	Puede duplicar datos	Normalizado, sin duplicar
Ideal para	Uno-a-pocos, datos que se leen juntos	Uno-a-muchos o muchos-a-muchos, datos independientes

Diseño propuesto para el blog

```
// users (autores)
{ _id, nombre, email, bio, avatar, createdAt }

// posts (publicaciones)
{ _id, titulo, slug, contenido, etiquetas: ['mongo','node'],
  autor: ObjectId('...users'),                        // REFERENCIA
  autorResumen: { nombre: 'Ana', avatar: '...' },    // EMBEBIDO (copia del dato leído
siempre)
  conteoComentarios: 42, publicadoEn, createdAt }

// comments (comentarios)
{ _id, post: ObjectId('...posts'), autor: ObjectId('...users'),
  autorResumen: { nombre: 'Luis' }, texto, createdAt }
```

Justificación

- **Autor en posts → referencia** (más un pequeño resumen embebido): un autor tiene muchas publicaciones y su perfil se edita por separado; el resumen (nombre y avatar) evita un populate al listar.
- **Etiquetas → embebidas** como arreglo: son pocas y siempre se leen con el post.
- **Comentarios → colección aparte (referencia al post):** pueden ser miles (crecimiento sin límite → riesgo de superar 16 MB), se paginan y se moderan de forma independiente. Un índice { post: 1, createdAt: -1 } hace rápida la consulta.

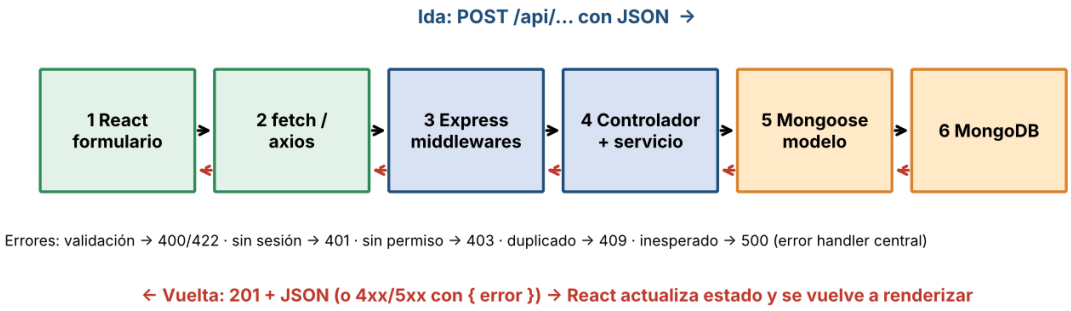
Ventajas y desventajas del diseño

- **Ventajas:** el listado de posts es rápido (sin joins), los comentarios escalan sin límite y los datos principales no se duplican, salvo un resumen pequeño.
- **Desventajas:** al cambiar el nombre de un autor hay que actualizar también los resúmenes embebidos (consistencia eventual); mostrar un post con comentarios requiere dos consultas; el contador conteoComentarios debe mantenerse con $inc al crear o borrar comentarios.

63. Describa el recorrido completo de una petición desde que el usuario pulsa "Guardar" en un formulario React hasta que el dato se persiste en MongoDB y la interfaz se actualiza. Mencione cada capa y lo que ocurre en ella, incluyendo el manejo de errores.

Respuesta 63

Escenario: el usuario llena un formulario de “Crear ticket” y pulsa **Guardar**. Esto es lo que ocurre, capa por capa:



1) React (cliente) – formulario

- El formulario es controlado (estado con useState). Al pulsar Guardar se ejecuta onSubmit, con **e.preventDefault()** para evitar que el navegador recargue la página.
- Se valida en el cliente (campos obligatorios, longitudes) y se muestra el estado “guardando” (botón deshabilitado).

2) Llamada HTTP (fetch)

```
const res = await fetch('/api/tickets', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  credentials: 'include',            // envía la cookie de sesión
  body: JSON.stringify(form),
});
```

- La petición viaja por la red (si el origen es distinto, el navegador aplica CORS y puede enviar antes una petición OPTIONS de verificación).

3) Express – middlewares y rutas

- Pasa por helmet, cors, express.json() (convierte el JSON en req.body), cookie-parser y el middleware **autenticar** (verifica el JWT → 401 si falla). Luego el router lo dirige al controlador: POST /api/tickets.

4) Controlador y servicio

- Se **valida** la entrada en el servidor (zod/joi: 400 o 422 si hay errores), se aplican reglas de negocio (por ejemplo, el cliente es el usuario autenticado, estado inicial “abierto”) y se llama al modelo.

5) Mongoose y MongoDB

- Ticket.create(datos): Mongoose valida el esquema, convierte tipos y envía un insertOne a MongoDB, que guarda el documento (BSON), aplica índices (por ejemplo, los únicos) y devuelve el documento con su _id.

6) Respuesta y actualización de la interfaz

- El controlador responde **201 Created** con el ticket en JSON. En React, el .then/await recibe la respuesta, actualiza el estado (por ejemplo, agrega el ticket a la lista o navega al detalle) y React **vuelve a renderizar** la pantalla con el nuevo dato.

7) Manejo de errores en cada capa

Capa	Error posible	Cómo se maneja
React	Datos inválidos / sin red	Validación previa; try/catch; mensaje al usuario; reactivar el botón
Express	JSON mal formado, sin sesión, sin permiso	400, 401 o 403 desde middlewares
Validación	Campos incorrectos	400/422 con lista de errores por campo, que el formulario muestra junto a cada input
Mongoose/MongoDB	Duplicado (E11000), CastError, caída de la base	409, 400 o 503 traducidos
Error handler central	Cualquier excepción	Formato uniforme { error, detalles }; se registra en el log y no se exponen trazas al cliente

---

64. Diseñe una estrategia de autenticación con JWT para una aplicación MERN usando access token y refresh token. Indique dónde se almacenan, cómo se renuevan, cómo se cierra sesión y qué riesgos de seguridad existen y cómo se mitigan.

Respuesta 64

Idea previa. Un **access token** es un JWT de **vida corta** (por ejemplo 15 minutos) que se envía en cada petición para demostrar quién eres. Si se roba, el daño dura poco. Como obligar al usuario a iniciar sesión

cada 15 minutos sería molesto, se usa un **refresh token** de **vida larga** (por ejemplo 7 días) cuya única función es obtener un nuevo access token.

1) Flujo propuesto

- **Login:** POST /api/auth/login → el servidor verifica email y contraseña (bcrypt), genera el access token (15 min) y el refresh token (7 días) y los envía en **cookies HttpOnly, Secure, SameSite**. El refresh token se guarda (su **hash**) en la base de datos con un identificador de familia/sesión.
- **Uso:** el navegador envía la cookie del access token automáticamente en cada petición; el middleware autenticar lo verifica.
- **Renovación:** cuando una petición recibe **401** por expiración, el cliente llama a POST /api/auth/refresh (la cookie del refresh token viaja sola, con Path limitado a esa ruta). El servidor la valida, comprueba que exista en la base, la **rota** (emite un refresh token nuevo e invalida el anterior) y emite un nuevo access token.
- **Logout:** POST /api/auth/logout → el servidor **revoca** (borra) el refresh token en la base y limpia ambas cookies (res.clearCookie). El access token expirará solo en pocos minutos.

```
// Emisión
const access  = jwt.sign({ id: u._id, rol: u.rol }, process.env.JWT_SECRET, { expiresIn: '15m'
});
const refresh = jwt.sign({ id: u._id, jti }, process.env.REFRESH_SECRET, { expiresIn: '7d' });

res.cookie('token', access,   { httpOnly: true, secure: true, sameSite: 'lax', maxAge:
15*60*1000 });
res.cookie('refresh', refresh, { httpOnly: true, secure: true, sameSite: 'strict',
                                 path: '/api/auth/refresh', maxAge: 7*24*3600*1000 });
```

2) Dónde se almacenan

Token	En el cliente	En el servidor
Access	Cookie HttpOnly (no accesible por JS)	No se guarda (es autoverificable por firma)
Refresh	Cookie HttpOnly con Path=/api/auth/refresh	Hash del token en la colección de sesiones (permite revocar)

3) Riesgos y mitigaciones

Riesgo	Mitigación
XSS roba el token	Cookies HttpOnly (no localStorage), CSP con helmet, escapar la entrada
CSRF (cookies enviadas desde otro sitio)	SameSite=Lax/Strict, token CSRF o verificar el origen en rutas que modifican datos
Robo del refresh token	Rotación en cada uso y **detección de reutilización**: si llega uno ya usado, se revoca toda la familia
Token robado en tránsito	HTTPS obligatorio y atributo Secure
Clave débil o algoritmo manipulado	Secreto largo y aleatorio en variables de entorno; fijar algorithms: ['HS256'] al verificar
Cambio de rol o usuario desactivado mientras el token sigue vigente	Vida corta del access token y consultar el usuario en la base en acciones sensibles
Fuerza bruta en login	Rate limiting y bloqueo temporal

---

65. Explique cuándo se vuelve a renderizar un componente funcional de React, qué fases ocurren y en qué casos debe usarse useEffect y en cuáles no. Dé un ejemplo de uso incorrecto y su corrección.

Respuesta 65

Idea previa. Un componente funcional de React es una **función** que recibe props y devuelve la interfaz (JSX). "Renderizar" es **volver a ejecutar esa función** para calcular qué debe verse.

1) ¿Cuándo se vuelve a renderizar?

- Cuando cambia su **estado** (useState/useReducer) con un valor distinto (React compara con Object.is).
- Cuando cambian sus **props** o, en general, cuando **su padre se vuelve a renderizar** (aunque las props sean iguales, salvo que use React.memo).
- Cuando cambia el valor de un **Context** que consume.
- No se re-renderiza por cambiar una variable normal, ni por mutar el estado sin llamar al setter, ni por modificar una ref.

2) Fases

- **Render:** React ejecuta la función del componente (debe ser **pura**: sin efectos secundarios) y calcula el nuevo árbol de elementos.
- **Reconciliación:** compara el árbol nuevo con el anterior (algoritmo "diffing", apoyado en las keys) y decide el **mínimo** de cambios.
- **Commit:** aplica solo esos cambios al DOM real.
- **Efectos:** después del commit se ejecutan los useEffect (primero se ejecuta la **limpieza** del efecto anterior si hay dependencias que cambiaron). Al desmontar, se ejecuta la limpieza final.

3) ¿Cuándo usar useEffect y cuándo no?

useEffect sirve para **sincronizar el componente con algo externo a React**:

- **Sí:** pedir datos al montar, suscribirse a eventos/WebSockets/timers (con limpieza), manipular el DOM manualmente, guardar en localStorage, integrar librerías externas.

- **No:** para **calcular valores derivados** del estado o las props (se calculan durante el render o con useMemo); para **responder a eventos del usuario** (va en el manejador onClick/onSubmit); para reiniciar estado cuando cambia una prop (usar key); ni para encadenar estados (un efecto que hace setState a partir de otro estado).

4) Ejemplo de uso incorrecto y su corrección

Incorrecto: se guarda en el estado algo que se puede calcular y se sincroniza con un efecto: provoca un render extra (y el efecto sin dependencias genera un bucle infinito).

```
function Carrito({ items }) {
  const [total, setTotal] = useState(0);
  useEffect(() => {
    setTotal(items.reduce((s, i) => s + i.precio, 0));   // render extra innecesario
  });                                                    // sin [] → bucle infinito
  return <p>Total: {total}</p>;
}
```

Corrección: calcular el valor directamente durante el render (con useMemo solo si el cálculo es costoso).

```
function Carrito({ items }) {
  const total = items.reduce((s, i) => s + i.precio, 0);   // valor derivado
  return <p>Total: {total}</p>;
}
```

Uso correcto de un efecto (datos externos, con limpieza y dependencias):

```
useEffect(() => {
  const ctrl = new AbortController();
  fetch(`/api/tickets/${id}`, { signal: ctrl.signal }).then(r => r.json()).then(setTicket);
  return () => ctrl.abort();        // limpieza al desmontar o cambiar id
}, [id]);
```

Parte II. Ejercicios prácticos por tecnología (30 puntos)

Cada ejercicio vale 3 puntos. Entregue cada solución en su archivo correspondiente dentro de la carpeta parte2 del repositorio. Se evalúa que el código sea correcto, legible, que maneje errores y que siga las buenas prácticas de cada tecnología.

Ejercicio P1 · JavaScript asíncrono (3 pts)

Implemente la función async obtenerUsuarios(ids, fetchUser, ms = 2000), donde fetchUser(id) devuelve una Promise con el usuario. La función debe:

- Ejecutar todas las peticiones en paralelo.
- No fallar si alguna petición falla.
- Aplicar un tiempo límite de ms milisegundos a cada petición.
- Devolver un objeto { ok: [usuarios], errores: [{ id, mensaje }] }.

Solución P1

Plan: (1) lanzar todas las peticiones a la vez con ids.map → paralelo; (2) envolver cada una en una promesa con **límite de tiempo** (Promise-race manual con setTimeout); (3) esperar con **Promise.allSettled**, que nunca se rechaza y reporta cada resultado; (4) separar éxitos y errores guardando el id.

```
const conTimeout = (promesa, ms, id) =>
  new Promise((resolve, reject) => {
    const timer = setTimeout(
      () => reject(new Error(`Tiempo agotado (${ms} ms) para el id ${id}`)), ms);
    promesa.then(
      (valor) => { clearTimeout(timer); resolve(valor); },
      (error) => { clearTimeout(timer); reject(error); }
    );
  });

export async function obtenerUsuarios(ids, fetchUser, ms = 2000) {
  const resultados = await Promise.allSettled(
    ids.map((id) =>
      // Promise.resolve().then(...) captura también errores síncronos de fetchUser
      conTimeout(Promise.resolve().then(() => fetchUser(id)), ms, id))
  );

  const salida = { ok: [], errores: [] };
  resultados.forEach((r, i) => {
    if (r.status === 'fulfilled') salida.ok.push(r.value);
    else salida.errores.push({ id: ids[i], mensaje: r.reason?.message ?? String(r.reason) });
  });
  return salida;
}
```

Por qué funciona: como allSettled espera a todas, una petición que falla o se pasa de tiempo solo se registra en errores y las demás continúan. clearTimeout evita temporizadores colgados. *Nota:* el límite de tiempo “abandona” la espera; para cancelar realmente la petición se necesitaría pasar un AbortSignal a fetchUser.

```
// Prueba rápida
const fake = (id) => id === 2 ? Promise.reject(new Error('Falla'))
  : id === 3 ? new Promise(() => {}) : Promise.resolve({ id });
console.log(await obtenerUsuarios([1, 2, 3], fake, 500));
```

```
// { ok: [ { id: 1 } ], errores: [ { id: 2, mensaje: 'Falla' },
//                                 { id: 3, mensaje: 'Tiempo agotado (500 ms) para el id 3' }
] }
```

Ejercicio P2 · Node.js: streams y sistema de archivos (3 pts)

Escriba el script contar.js que reciba por línea de comandos la ruta de un archivo CSV (puede pesar más de 2 GB) con el encabezado id,categoria,monto. Debe mostrar, por categoría, el número de registros y la suma de los montos, sin cargar el archivo completo en memoria, ignorando líneas vacías o con montos inválidos. Uso: node contar.js ventas.csv

Solución P2

Plan: un archivo de 2 GB no cabe cómodamente en memoria, por lo que se lee como **flujo (stream)**: se procesa **línea por línea** con readline sobre fs.createReadStream y solo se guarda en memoria un pequeño Map con los totales por categoría (muy pocos datos). Se ignoran líneas vacías y montos no numéricos.

```
// contar.js  (CommonJS: node contar.js ventas.csv)
const fs = require('node:fs');
const readline = require('node:readline');

async function main() {
  const ruta = process.argv[2];
  if (!ruta) {
    console.error('Uso: node contar.js ventas.csv');
    process.exit(1);
  }

  const flujo = fs.createReadStream(ruta, { encoding: 'utf8' });
  flujo.on('error', (e) => {
    console.error(`No se pudo leer "${ruta}": ${e.message}`);
    process.exit(1);
  });
  const rl = readline.createInterface({ input: flujo, crlfDelay: Infinity });

  const totales = new Map();           // categoria -> { registros, suma }
  let primera = true;

  for await (const linea of rl) {
    if (primera) { primera = false; continue; }      // salta el encabezado
    const texto = linea.trim();
    if (!texto) continue;                            // línea vacía

    const [, categoria, monto] = texto.split(',');
    const valor = Number(monto);
    if (!categoria?.trim() || monto === undefined || monto.trim() === ''
        || !Number.isFinite(valor)) continue;        // monto inválido

    const clave = categoria.trim();
    const acc = totales.get(clave) ?? { registros: 0, suma: 0 };
    acc.registros += 1;
    acc.suma += valor;
    totales.set(clave, acc);
  }

  for (const [cat, { registros, suma }] of [...totales].sort()) {
    console.log(`${cat}\t${registros}\t${suma.toFixed(2)}`);
  }
}
```

```
main();
```

Justificación: la memoria usada depende del número de **categorías**, no del tamaño del archivo (O(1) respecto a las filas). El flujo respeta la "contrapresión" (lee a la velocidad que se procesa). Si el CSV usa comillas o comas dentro de los campos, convendría un parser como csv-parse; aquí se asume el formato simple pedido.

Ejercicio P3 · Express: depuración de una API (3 pts)

El siguiente servidor tiene varios errores. Identifique al menos seis, explique cada uno y entregue la versión corregida.

```
const express = require('express');
const app = express();

app.get('/api/productos/:id', async (req, res) => {
  const p = await Producto.findById(req.params.id);
  if (!p) res.status(404).json({ error: 'No encontrado' });
  res.json(p);
});

app.get('/api/productos/destacados', async (req, res) => {
  res.json(await Producto.find({ destacado: true }));
});

app.post('/api/productos', async (req, res) => {
  const p = await Producto.create(req.body);
  res.json(p);
});

app.use((err, req, res) => {
  res.status(500).send(err.message);
});

app.listen(3000);
```

Solución P3

Errores encontrados

N.º	Error	Consecuencia
1	Falta **return** en if (!p) res.status(404)…	Después de responder 404 sigue y ejecuta res.json(p) → "Cannot set headers after they are sent" (y envía null).
2	La ruta dinámica **/:id** está antes que **/destacados**	GET /api/productos/destacados entra a /:id con id = "destacados" → CastError / 500. Las rutas específicas van primero.
3	No se usa **express.json()**	req.body es undefined en el POST: Producto.create(undefined) falla o crea documentos vacíos.
4	El POST responde **200** en lugar de **201 Created**	Código HTTP incorrecto para la creación de un recurso.
5	El middleware de errores tiene **3 parámetros** (err, req, res)	Express no lo reconoce como manejador de errores (necesita 4: err, req, res, next); los errores usan el manejador por defecto.
6	Se responde **err.message** al cliente	Filtra información interna (detalles de la base, trazas). Debe devolverse un mensaje genérico y registrar el error en el log.

N.º	Error	Consecuencia
7	Id sin validar: findById con un id mal formado	Lanza CastError → 500. Debe validarse con mongoose.isValidObjectId y responder 400.
8	**Producto.create(req.body)** (asignación masiva)	El cliente puede enviar campos no permitidos (precio, destacado, _id…). Se debe validar y filtrar los campos permitidos.
9	Faltan el **require del modelo** y la **conexión a MongoDB**	Producto no está definido y la aplicación escucha sin tener base de datos. Hay que conectar con mongoose.connect antes de app.listen.
10	Sin manejo de **rutas inexistentes** y **puerto fijo (3000)**	Las URL desconocidas responden con el 404 HTML por defecto; el puerto debe venir de process.env.PORT.

Versión corregida

```
const express = require('express');
const mongoose = require('mongoose');
const Producto = require('./models/Producto');   // modelo importado

const app = express();
app.use(express.json());                          // (3)

// (2) la ruta específica va ANTES de la dinámica
app.get('/api/productos/destacados', async (req, res) => {
  res.json(await Producto.find({ destacado: true }));
});

app.get('/api/productos/:id', async (req, res) => {
  if (!mongoose.isValidObjectId(req.params.id)) {          // (7)
    return res.status(400).json({ error: 'Id inválido' });
  }
  const p = await Producto.findById(req.params.id);
  if (!p) return res.status(404).json({ error: 'No encontrado' });   // (1) return
  res.json(p);
});

app.post('/api/productos', async (req, res) => {
  const { nombre, precio, stock } = req.body ?? {};        // (8) solo campos permitidos
  const p = await Producto.create({ nombre, precio, stock });
  res.status(201).json(p);                                  // (4)
});

app.use((req, res) => res.status(404).json({ error: 'Ruta no encontrada' })); // (10)

app.use((err, req, res, next) => {                          // (5) cuatro parámetros
  console.error(err);                                       // (6) se registra, no se expone
  if (err.name === 'ValidationError') return res.status(400).json({ error: err.message });
  res.status(err.status ?? 500).json({ error: 'Error interno del servidor' });
});

const PORT = process.env.PORT || 3000;
mongoose.connect(process.env.MONGODB_URI)                   // (9)
  .then(() => app.listen(PORT, () => console.log(`API en puerto ${PORT}`)))
  .catch((e) => { console.error(e); process.exit(1); });
```

Nota: en Express 5 los errores de los handlers async llegan solos al manejador de errores. En Express 4 habría que envolver cada handler en try/catch con next(err).

Ejercicio P4 · Express: autenticación y autorización con middlewares (3 pts)

Implemente dos middlewares:

- autenticar: lee el JWT de la cookie token, lo verifica con jsonwebtoken y el secreto de process.env.JWT_SECRET, y adjunta req.user = { id, rol }. Responde 401 si falta o es inválido.
- autorizar(...roles): responde 403 si el rol de req.user no está entre los permitidos.

Muestre su uso en una ruta DELETE /api/usuarios/:id accesible solo para administradores.

Solución P4

Idea previa. Autenticar = comprobar *quién eres* (verificar el token) → falla con **401**. **Autorizar** = comprobar *qué puedes hacer* (tu rol) → falla con **403**. Se necesita cookie-parser para que Express lea req.cookies.

```
// middlewares/auth.js
import jwt from 'jsonwebtoken';

export function autenticar(req, res, next) {
  const token = req.cookies?.token;
  if (!token) return res.status(401).json({ error: 'No autenticado' });
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET, { algorithms: ['HS256'] });
    req.user = { id: payload.id, rol: payload.rol };
    next();
  } catch {
    return res.status(401).json({ error: 'Token inválido o expirado' });
  }
}

// Fábrica de middleware: devuelve la función con los roles permitidos
export const autorizar = (...roles) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.rol)) {
    return res.status(403).json({ error: 'No tiene permiso para esta acción' });
  }
  next();
};
```

```
// app.js / routes/usuarios.js
import cookieParser from 'cookie-parser';
app.use(cookieParser());

router.delete('/api/usuarios/:id', autenticar, autorizar('admin'), async (req, res) => {
  if (!mongoose.isValidObjectId(req.params.id)) {
    return res.status(400).json({ error: 'Id inválido' });
  }
  const borrado = await User.findByIdAndDelete(req.params.id);
  if (!borrado) return res.status(404).json({ error: 'Usuario no encontrado' });
  res.status(204).end();
});
```

Orden importante: primero autenticar (llena req.user) y después autorizar (lo usa). Como el rol viene dentro del token firmado, nadie puede cambiarlo sin invalidar la firma. Para acciones sensibles puede además consultarse la base de datos, por si el usuario fue desactivado después de emitir el token.

Ejercicio P5 · MongoDB: consultas, agregación e índices (3 pts)

La colección orders tiene documentos con la siguiente forma:

```
{
  _id: ObjectId('...'),
  customerId: ObjectId('...'),
  status: 'paid',            // 'pending' | 'paid' | 'shipped' | 'cancelled'
  total: 1530.5,
  items: [ { sku: 'KB-01', qty: 2, price: 450 }, { sku: 'MS-07', qty: 1, price: 630.5 } ],
  createdAt: ISODate('2026-03-14T10:22:00Z')
}
```

Escriba en mongosh:

- a) Los 10 pedidos más recientes con status 'paid' y total mayor a 1000, mostrando solo customerId, total y createdAt.
- b) Para 2026, el número de pedidos pagados y el ingreso total por mes, ordenado por mes.
- c) Los 5 SKU con más unidades vendidas en pedidos pagados, con su ingreso.
- d) El índice que optimiza la consulta del inciso a), con su justificación y cómo verificaría que se usa.
- e) Una actualización que marque como 'cancelled' los pedidos 'pending' con más de 30 días de antigüedad, registrando cancelledAt.

Solución P5

a) Los 10 pedidos pagados más recientes con total > 1000

```
db.orders.find(
  { status: 'paid', total: { $gt: 1000 } },
  { customerId: 1, total: 1, createdAt: 1, _id: 0 }     // proyección
).sort({ createdAt: -1 }).limit(10)
```

b) Pedidos pagados e ingreso por mes en 2026

```
db.orders.aggregate([
  { $match: { status: 'paid',
              createdAt: { $gte: ISODate('2026-01-01'), $lt: ISODate('2027-01-01') } } },
  { $group: { _id: { $dateToString: { format: '%Y-%m', date: '$createdAt' } },
              pedidos: { $sum: 1 },
              ingreso: { $sum: '$total' } } },
  { $sort: { _id: 1 } }
])
```

$match filtra primero (para poder usar índice); $group agrupa por mes y suma; $sort ordena por mes (el formato AAAA-MM se ordena bien como texto).

c) Top 5 SKU por unidades vendidas (pedidos pagados), con ingreso

```
db.orders.aggregate([
  { $match: { status: 'paid' } },
  { $unwind: '$items' },                        // un documento por cada item
  { $group: { _id: '$items.sku',
              unidades: { $sum: '$items.qty' },
              ingreso: { $sum: { $multiply: ['$items.qty', '$items.price'] } } } },
  { $sort: { unidades: -1 } },
  { $limit: 5 }
])
```

d) Índice para la consulta del inciso a)

```
db.orders.createIndex({ status: 1, createdAt: -1, total: 1 })
```

Justificación (regla ESR): *Equality* primero: status (igualdad); *Sort* después: createdAt (el orden coincide con el del sort, así no hace falta ordenar en memoria); *Range* al final: total (rango $gt). Con este índice MongoDB recorre solo los pedidos "paid" ya ordenados y se detiene al encontrar 10.

Cómo verificar que se usa:

```
db.orders.find({ status: 'paid', total: { $gt: 1000 } })
  .sort({ createdAt: -1 }).limit(10).explain('executionStats')
```

- En winningPlan debe aparecer **IXSCAN** (con el nombre del índice) y **no** COLLSCAN ni una etapa SORT.
- totalDocsExamined debe ser cercano a nReturned (≈ 10) y totalKeysExamined bajo.

e) Cancelar pedidos pendientes con más de 30 días

```
db.orders.updateMany(
  { status: 'pending',
    createdAt: { $lt: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000) } },
  { $set: { status: 'cancelled' },
    $currentDate: { cancelledAt: true } }          // registra la fecha actual
)
```

updateMany aplica el cambio a todos los pedidos que cumplan el filtro; la fecha límite se calcula como "ahora menos 30 días" (en milisegundos).

Ejercicio P6 · Mongoose: modelado y validación (3 pts)

Defina el modelo User con:

- nombre obligatorio (máximo 80 caracteres, sin espacios sobrantes).
- email obligatorio, único, en minúsculas y con formato válido.
- password obligatorio con al menos 8 caracteres, almacenado con hash bcrypt y que no se devuelva por defecto en las consultas.
- rol con los valores cliente, agente o admin (cliente por defecto).
- Marcas de tiempo automáticas.
- Un método de instancia compararPassword(candidata).
- Que la serialización a JSON nunca incluya password ni __v.

Solución P6

Idea previa. Un **esquema** de Mongoose describe qué campos tiene un documento y qué reglas cumplen. La **contraseña** nunca se guarda en claro: un *hook* **pre('save')** la convierte en hash con bcrypt justo antes de guardar. Con select: false la contraseña no se devuelve en las consultas salvo que se pida con .select('+password').

```
// models/User.js
import mongoose from 'mongoose';
import bcrypt from 'bcrypt';

const userSchema = new mongoose.Schema(
  {
    nombre: {
      type: String,
      required: [true, 'El nombre es obligatorio'],
      trim: true,
      maxlength: [80, 'El nombre no puede pasar de 80 caracteres'],
```

```
    },
    email: {
      type: String,
      required: [true, 'El email es obligatorio'],
      unique: true,                  // crea un índice único en MongoDB
      lowercase: true,
      trim: true,
      match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, 'Email con formato inválido'],
    },
    password: {
      type: String,
      required: [true, 'La contraseña es obligatoria'],
      minlength: [8, 'La contraseña debe tener al menos 8 caracteres'],
      select: false,                 // no se devuelve por defecto
    },
    rol: {
      type: String,
      enum: { values: ['cliente', 'agente', 'admin'], message: 'Rol inválido: {VALUE}' },
      default: 'cliente',
    },
  },
  {
    timestamps: true,                // createdAt y updatedAt
    toJSON: {
      versionKey: false,             // quita __v
      transform: (doc, ret) => { delete ret.password; return ret; },
    },
  }
);

// Hash solo si la contraseña se creó o cambió
userSchema.pre('save', async function () {
  if (!this.isModified('password')) return;
  this.password = await bcrypt.hash(this.password, 12);
});

// Método de instancia: compara la contraseña en claro con el hash
userSchema.methods.compararPassword = function (candidata) {
  return bcrypt.compare(candidata, this.password);
};

export default mongoose.model('User', userSchema);
```

```
// Uso en el login (hay que pedir explícitamente el campo oculto)
const user = await User.findOne({ email }).select('+password');
if (!user || !(await user.compararPassword(password))) {
  return res.status(401).json({ error: 'Credenciales inválidas' });   // mismo mensaje en
ambos casos
}
```

Detalles clave: (1) el orden: las validaciones (minlength) se ejecutan **antes** del hook, sobre la contraseña en claro, y luego se hashea; (2) isModified evita volver a hashear un hash cuando se edita otro campo; (3) toJSON elimina password y __v al responder; (4) el mensaje de login es el mismo para “usuario inexistente” y “contraseña incorrecta” para no revelar qué emails existen.

Ejercicio P7 · React: depuración de un componente (3 pts)

El siguiente componente presenta varios errores. Identifíquelos, explique sus consecuencias y entregue una versión corregida.

```
function ListaTickets() {
```

```
  const [tickets, setTickets] = useState([]);
  const [filtro, setFiltro] = useState('');

  useEffect(() => {
    fetch('/api/tickets')
      .then(r => r.json())
      .then(data => setTickets(data));
  });

  const cerrar = (id) => {
    const t = tickets.find(t => t._id === id);
    t.estado = 'cerrado';
    setTickets(tickets);
  };

  return (
    <>
      <input value={filtro} />
      {tickets.filter(t => t.titulo.includes(filtro)).map((t, i) => (
        <div key={i}>
          {t.titulo} - {t.estado}
          <button onClick={cerrar(t._id)}>Cerrar</button>
        </div>
      ))}
    </>
  );
}
```

Solución P7

Errores encontrados

N.º	Error	Por qué es un problema
1	**useEffect sin arreglo de dependencias**	Se ejecuta tras cada render: el fetch actualiza el estado → nuevo render → nuevo fetch… **bucle infinito**. Debe llevar [].
2	No hay manejo de **errores ni de carga** (r.ok, catch, loading)	Si falla la red o el servidor, la pantalla queda vacía o rompe sin avisar al usuario.
3	**Muta el estado**: t.estado = 'cerrado' y setTickets(tickets)	Se cambia el objeto original y se pasa la misma referencia: React cree que nada cambió y no vuelve a renderizar. Además no guarda el cambio en el servidor.
4	Input con value={filtro} **sin onChange**	Campo de solo lectura (React advierte): el usuario no puede escribir; el filtro nunca cambia.
5	key={i} (índice del arreglo)	Al filtrar o reordenar se mezclan estados e identidades; debe usarse el id único (t._id).
6	onClick={cerrar(t._id)}	Llama a cerrar **durante el render**, no al hacer clic (provoca actualizaciones en cadena). Debe ser onClick={() => cerrar(t._id)}.
7	t.titulo.includes(filtro) distingue mayúsculas y puede fallar si titulo es undefined	El filtro no encuentra “Error” al buscar “error” y lanza TypeError si falta el título.
8	Faltan los **imports** de useState y useEffect y un estado vacío	El componente ni siquiera compila; no se informa cuando no hay resultados.

Componente corregido

```
import { useState, useEffect } from 'react';

export default function ListaTickets() {
  const [tickets, setTickets] = useState([]);
  const [filtro, setFiltro] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {                                    // (1) [] → solo al montar
    const ctrl = new AbortController();
    (async () => {
      try {
        const r = await fetch('/api/tickets', { signal: ctrl.signal, credentials: 'include'
});
        if (!r.ok) throw new Error(`Error HTTP ${r.status}`);       // (2)
        setTickets(await r.json());
      } catch (e) {
        if (e.name !== 'AbortError') setError(e.message);
      } finally {
        if (!ctrl.signal.aborted) setLoading(false);
      }
    })();
    return () => ctrl.abort();                         // limpieza al desmontar
  }, []);

  const cerrar = async (id) => {
    const r = await fetch(`/api/tickets/${id}`, {
      method: 'PATCH', credentials: 'include',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ estado: 'cerrado' }),
    });
    if (r.ok) {                                        // (3) inmutable: nuevo arreglo y nuevo
objeto
      setTickets((prev) => prev.map((t) => (t._id === id ? { ...t, estado: 'cerrado' } : t)));
    }
  };

  const visibles = tickets.filter((t) =>             // (7)
    (t.titulo ?? '').toLowerCase().includes(filtro.toLowerCase()));

  if (loading) return <p>Cargando…</p>;
  if (error) return <p role="alert">No se pudieron cargar los tickets: {error}</p>;

  return (
    <div>
      <input value={filtro} onChange={(e) => setFiltro(e.target.value)}   // (4)
             placeholder="Buscar…" />
      {visibles.length === 0 && <p>No hay tickets.</p>}                    {/* (8) */}
      <ul>
        {visibles.map((t) => (
          <li key={t._id}>                                                    {/* (5) */}
            {t.titulo} — {t.estado}
            <button onClick={() => cerrar(t._id)}>Cerrar</button>             {/* (6) */}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Ejercicio P8 · React: custom hook useFetch (3 pts)

Implemente el hook useFetch(url) que devuelva { data, loading, error, recargar }. Debe cancelar la petición en curso si la url cambia o el componente se desmonta, tratar como error las respuestas HTTP no exitosas y permitir volver a ejecutar la petición con recargar(). Muestre un ejemplo de uso.

Solución P8

Idea previa. Un **custom hook** es una función cuyo nombre empieza con "use" y que agrupa lógica con hooks (estado, efectos) para **reutilizarla** en varios componentes. Aquí encapsula el patrón "pedir datos → mostrar cargando → manejar error". La cancelación con **AbortController** evita que una respuesta tardía de una URL anterior sobrescriba el estado, o actualice un componente ya desmontado.

```
// hooks/useFetch.js
import { useState, useEffect, useCallback } from 'react';

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(Boolean(url));
  const [error, setError] = useState(null);
  const [version, setVersion] = useState(0);     // al cambiar, se vuelve a pedir

  const recargar = useCallback(() => setVersion((v) => v + 1), []);

  useEffect(() => {
    if (!url) { setLoading(false); return; }
    const controller = new AbortController();
    setLoading(true);
    setError(null);

    (async () => {
      try {
        const res = await fetch(url, { signal: controller.signal, credentials: 'include' });
        if (!res.ok) throw new Error(`Error HTTP ${res.status}`);
        setData(await res.json());
      } catch (e) {
        if (e.name === 'AbortError') return;        // petición cancelada: se ignora
        setError(e);
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    })();

    return () => controller.abort();              // al cambiar url o desmontar
  }, [url, version]);

  return { data, loading, error, recargar };
}
```

```
// Ejemplo de uso
function Productos() {
  const { data, loading, error, recargar } = useFetch('/api/productos');
  if (loading) return <p>Cargando…</p>;
  if (error) return <p>Error: {error.message} <button
onClick={recargar}>Reintentar</button></p>;
  return <ul>{data.items.map((p) => <li key={p._id}>{p.nombre}</li>)}</ul>;
}
```

Ejercicio P9 · React: formulario controlado y manejo de listas (3 pts)

Construya el componente GestorTareas con:

- Un input controlado y un botón para agregar tareas (no se permiten tareas vacías; se eliminan los espacios sobrantes).
- Una lista con checkbox para marcar cada tarea como completada y un botón para eliminarla.
- Un contador de tareas pendientes.
- Un filtro con las opciones Todas, Pendientes y Completadas.

Requisitos: no mutar el estado, usar keys estables y que el formulario funcione al presionar Enter.

Solución P9

Plan: (1) estado con la lista de tareas (id, texto, completada), el texto del input y el filtro activo; (2) un **<form>** con onSubmit para que Enter funcione y para evitar tareas vacías; (3) todas las actualizaciones **inmutables** (map, filter y spread); (4) el contador de pendientes y la lista filtrada se **calculan durante el render**, sin estado extra ni useEffect.

```
import { useState } from 'react';

export default function GestorTareas() {
  const [tareas, setTareas] = useState([]);
  const [texto, setTexto] = useState('');
  const [filtro, setFiltro] = useState('todas');   // 'todas' | 'pendientes' | 'completadas'

  const agregar = (e) => {
    e.preventDefault();
    const limpio = texto.trim();
    if (!limpio) return;                            // no acepta tareas vacías
    setTareas((prev) => [...prev, { id: crypto.randomUUID(), texto: limpio, completada: false
}]);
    setTexto('');
  };

  const alternar = (id) =>
    setTareas((prev) => prev.map((t) => (t.id === id ? { ...t, completada: !t.completada } :
t)));

  const eliminar = (id) => setTareas((prev) => prev.filter((t) => t.id !== id));

  // Valores derivados: se calculan, no se guardan en estado
  const pendientes = tareas.filter((t) => !t.completada).length;
  const visibles = tareas.filter((t) =>
    filtro === 'todas' ? true : filtro === 'pendientes' ? !t.completada : t.completada);

  return (
    <section>
      <h2>Mis tareas</h2>
      <form onSubmit={agregar}>
        <input value={texto} onChange={(e) => setTexto(e.target.value)} placeholder="Nueva
tarea" />
        <button type="submit">Agregar</button>
      </form>

      <div>
        {['todas', 'pendientes', 'completadas'].map((f) => (
          <button key={f} onClick={() => setFiltro(f)} aria-pressed={filtro ===
f}>{f}</button>
        ))}
      </div>
```

```
      <p>{pendientes} pendiente(s)</p>
      {visibles.length === 0 && <p>No hay tareas para mostrar.</p>}
      <ul>
        {visibles.map((t) => (
          <li key={t.id}>
            <label style={{ textDecoration: t.completada ? 'line-through' : 'none' }}>
              <input type="checkbox" checked={t.completada} onChange={() => alternar(t.id)} />
              {t.texto}
            </label>
            <button onClick={() => eliminar(t.id)}>Eliminar</button>
          </li>
        ))}
      </ul>
    </section>
  );
}
```

Ejercicio P10 · Integración: listado paginado de extremo a extremo (3 pts)

Implemente:

- En Express: GET /api/productos con los parámetros page (1 por defecto), limit (10 por defecto, máximo 100) y search (búsqueda por nombre sin distinguir mayúsculas). Debe responder { items, page, limit, total, totalPages }.
- En React: el componente CatalogoProductos que consuma el endpoint, con caja de búsqueda con debounce de 400 ms, botones Anterior y Siguiente, y estados de carga y error. Puede reutilizar el hook del ejercicio P8.

Solución P10

Parte 1 – Endpoint GET /api/productos (Express + Mongoose)

Idea: paginación = devolver los datos por “páginas” (page y limit) con skip/limit; se **limitan** los valores (limit máx. 100) para que nadie pida millones de registros; la **búsqueda** usa una expresión regular **escapada** (si no, un usuario podría enviar una regex maliciosa) y los parámetros se verifican como texto (evita inyección con ?search[$ne]=x).

```
const escapeRegex = (s) => s.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

router.get('/', async (req, res) => {
  const page = Math.max(parseInt(req.query.page, 10) || 1, 1);
  const limit = Math.min(Math.max(parseInt(req.query.limit, 10) || 10, 1), 100);
  const search = typeof req.query.search === 'string' ? req.query.search.trim() : '';

  const filtro = search ? { nombre: { $regex: escapeRegex(search), $options: 'i' } } : {};

  const [items, total] = await Promise.all([
    Producto.find(filtro).sort({ _id: 1 }).skip((page - 1) * limit).limit(limit).lean(),
    Producto.countDocuments(filtro),
  ]);

  res.json({ items, page, limit, total, totalPages: Math.ceil(total / limit) });
});
```

Nota de rendimiento: skip se vuelve lento con páginas muy profundas; para catálogos enormes se prefiere paginación por cursor (por ejemplo, { _id: { $gt: ultimoId } }).

Parte 2 – Componente CatalogoProductos con debounce (React)

Debounce = esperar a que el usuario **deje de escribir** (por ejemplo 400 ms) antes de lanzar la búsqueda, para no hacer una petición por cada tecla. Al cambiar el texto se reinicia la página a 1 **dentro del manejador del evento** (no con un useEffect).

```
import { useState, useEffect } from 'react';
import { useFetch } from './hooks/useFetch';

function useDebounce(valor, ms = 400) {
  const [debounced, setDebounced] = useState(valor);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(valor), ms);
    return () => clearTimeout(t);              // reinicia el temporizador en cada tecla
  }, [valor, ms]);
  return debounced;
}

export default function CatalogoProductos() {
  const [busqueda, setBusqueda] = useState('');
  const [page, setPage] = useState(1);
  const buscar = useDebounce(busqueda, 400);

  const qs = new URLSearchParams({ page, limit: 10, search: buscar });
  const { data, loading, error } = useFetch(`/api/productos?${qs}`);

  const onBuscar = (e) => { setBusqueda(e.target.value); setPage(1); };

  return (
    <section>
      <input value={busqueda} onChange={onBuscar} placeholder="Buscar producto…" />

      {loading && <p>Cargando…</p>}
      {error && <p role="alert">Error: {error.message}</p>}
      {data && data.items.length === 0 && <p>Sin resultados.</p>}

      <ul>
        {data?.items.map((p) => <li key={p._id}>{p.nombre} — ${p.precio}</li>)}
      </ul>

      {data && (
        <nav>
          <button disabled={page <= 1} onClick={() => setPage((p) => p - 1)}>Anterior</button>
          <span> Página {data.page} de {data.totalPages || 1} </span>
          <button disabled={page >= data.totalPages} onClick={() => setPage((p) => p +
1)}>Siguiente</button>
        </nav>
      )}
    </section>
  );
}
```

Parte III. Proyecto integrador: "HelpDesk MERN" (30 puntos)

Contexto

Una empresa necesita un sistema web de mesa de ayuda (help desk) para que sus clientes reporten incidencias y el equipo de soporte las atienda. Usted debe construir la aplicación completa con el stack MERN, demostrando dominio de cada tecnología y de su integración.

Roles

Rol	Descripción
Cliente	Crea tickets, consulta y comenta solo los suyos, puede cerrar o reabrir un ticket resuelto.
Agente	Atiende tickets: ve los que tiene asignados y los no asignados, se los autoasigna, cambia su estado, comenta y agrega notas internas.
Administrador	Ve todo, asigna tickets a agentes, gestiona usuarios (rol y activación) y consulta el panel de métricas.

Requerimientos funcionales

- **RF1. Autenticación:** registro de clientes, inicio y cierre de sesión con JWT en cookie HttpOnly, y endpoint /api/auth/me para recuperar la sesión.
- **RF2. Creación de tickets:** título, descripción, categoría (hardware, software, red, cuenta, otro) y prioridad (baja, media, alta, crítica).
- **RF3. Visibilidad por rol:** cada rol ve únicamente los tickets que le corresponden según la tabla anterior; la restricción se aplica en el servidor.
- **RF4. Flujo de estados:** abierto → en_progreso → resuelto → cerrado, con reapertura (resuelto → abierto) por el cliente. Las transiciones se validan en el servidor según el rol y cada cambio se registra en un historial (estado anterior, nuevo, usuario y fecha).
- **RF5. Comentarios:** clientes y agentes comentan los tickets; los agentes pueden marcar un comentario como nota interna, invisible para el cliente.
- **RF6. Asignación:** el administrador asigna tickets a agentes; un agente puede autoasignarse un ticket no asignado.
- **RF7. Listado:** paginación, filtros por estado, prioridad y categoría, búsqueda de texto y ordenamiento.
- **RF8. Panel de métricas** (agente y administrador): tickets por estado y por prioridad, tiempo promedio de resolución y carga por agente, calculados con pipelines de agregación de MongoDB.
- **RF9. Gestión de usuarios** (administrador): listar usuarios, cambiar rol y activar o desactivar cuentas.

Requerimientos no funcionales

- **RNF1. Validación** en cliente y servidor (zod, joi o express-validator) con mensajes de error claros.
- **RNF2. Seguridad:** contraseñas con bcrypt, helmet, CORS restringido al origen del frontend, rate limiting en el login, protección contra inyección NoSQL y ninguna exposición de trazas en producción.
- **RNF3. Arquitectura en capas** en el backend (rutas, controladores, servicios, modelos, middlewares) con manejo centralizado de errores y formato de error uniforme.

- **RNF4. Rendimiento:** índices adecuados para las consultas principales, justificados en el README.
- **RNF5. Pruebas:** al menos 8 pruebas de integración de la API (Supertest y mongodb-memory-server) y 3 pruebas de componentes (React Testing Library).
- **RNF6. Configuración** por variables de entorno, con .env.example.
- **RNF7. Interfaz** responsiva, con estados de carga, error y vacío, rutas protegidas por rol y accesibilidad básica (etiquetas, foco, contraste).
- **RNF8. Control de versiones** en Git con commits frecuentes y descriptivos.

Endpoints mínimos de la API

Método	Ruta	Acceso	Descripción
POST	/api/auth/register	Público	Registro de cliente
POST	/api/auth/login	Público	Inicio de sesión
POST	/api/auth/logout	Autenticado	Cierre de sesión
GET	/api/auth/me	Autenticado	Usuario de la sesión
GET	/api/tickets	Autenticado	Listado paginado y filtrado según rol
POST	/api/tickets	Cliente, admin	Crear ticket
GET	/api/tickets/stats	Agente, admin	Métricas del panel
GET	/api/tickets/:id	Autenticado (visibilidad)	Detalle con historial
PATCH	/api/tickets/:id/estado	Según transición	Cambiar estado
PATCH	/api/tickets/:id/asignar	Agente, admin	Asignar o autoasignar
GET	/api/tickets/:id/comentarios	Autenticado (visibilidad)	Comentarios (sin notas internas para el cliente)
POST	/api/tickets/:id/comentarios	Autenticado (visibilidad)	Agregar comentario
GET	/api/users	Admin	Listar usuarios
PATCH	/api/users/:id	Admin	Cambiar rol o estado activo

Vistas mínimas del frontend

- Inicio de sesión y registro.
- Listado de tickets con filtros, búsqueda y paginación.
- Formulario de creación de ticket.
- Detalle del ticket: datos, historial de estados, comentarios y acciones disponibles según el rol y el estado.
- Panel de métricas con al menos dos gráficas (por ejemplo, con Recharts o Chart.js).
- Administración de usuarios.

Entregables

- Repositorio Git con las carpetas server y client (o un monorepo equivalente).

- README con: descripción, diagrama de arquitectura, modelo de datos justificado (qué se embebe y qué se referencia, y por qué), índices creados, instrucciones de instalación y ejecución, variables de entorno, usuarios de prueba y comandos para ejecutar las pruebas.
- Script de datos semilla (seed) con al menos 3 clientes, 2 agentes, 1 administrador y 30 tickets.
- Colección de Postman o Bruno, o documentación OpenAPI de la API.
- Video de demostración de 5 a 8 minutos que recorra los tres roles.

Elementos opcionales (hasta 5 puntos extra, sin exceder 100)

- Docker Compose para levantar MongoDB, API y frontend.
- Despliegue público (por ejemplo, MongoDB Atlas con Render o Railway para la API y Vercel o Netlify para el frontend).
- TypeScript en el backend, el frontend o ambos.
- Notificaciones en tiempo real con Socket.IO cuando cambia un ticket.
- Integración continua con GitHub Actions que ejecute lint y pruebas.
- Refresh tokens con rotación.

Defensa oral (20 minutos)

El sustentante presentará su solución y responderá preguntas como las siguientes:

- ¿Qué decidió embeber y qué referenciar en su modelo, y por qué?
- Muestre el resultado de explain() de la consulta del listado. ¿Qué índice usa?
- ¿Qué ocurre si un usuario modifica el rol dentro de su JWT?
- ¿Cómo garantiza que un cliente no pueda ver ni modificar tickets ajenos, aunque conozca su id?
- Demuestre de extremo a extremo cómo un error de validación del servidor se muestra en la interfaz.
- Si el panel de métricas se volviera lento con un millón de tickets, ¿qué haría?
- Realice en vivo un cambio pequeño solicitado por el evaluador (por ejemplo, agregar un filtro nuevo).

Respuestas modelo a las preguntas de defensa oral

Estas son respuestas de referencia, que deben adaptarse a la implementación real del proyecto (la propuesta técnica completa está al final del documento).

- **Cómo evita que un cliente vea tickets ajenos.** La regla se aplica **en el servidor**, no en el frontend: un servicio construye un filtro según el rol del token (cliente → { cliente: req.user.id }; agente → { agente: req.user.id } o sin asignar; admin → sin restricción) y se combina con { _id } en el detalle. Si el ticket existe pero no pertenece al usuario se responde **404** (no 403) para no revelar que existe. Se prueba con pruebas automáticas (supertest) que intentan acceder con otro usuario.
- **Dónde se valida cada transición de estado.** En una **capa de servicio** del backend, con un mapa de transiciones permitidas por rol (abierto → en_progreso → resuelto → cerrado; reapertura solo por cliente/agente). El controlador solo la invoca; el frontend únicamente oculta botones por comodidad, pero **nunca** es la defensa. Si la transición no es válida, se responde 409 o 422 y no se guarda nada.
- **Embeber vs. referenciar comentarios e historial.** El **historial** de estados se **embebe** en el ticket (pocas entradas, siempre se lee con el ticket, se actualiza con el mismo cambio de forma atómica). Los **comentarios** van en **otra colección** (referencia al ticket) porque pueden crecer sin límite, se paginan y se moderan aparte; así se evita superar los 16 MB y cargar todo en cada lectura.
- **Índices creados y consultas que aceleran.** Ver la tabla de índices de la propuesta: { cliente: 1, createdAt: -1 } (mis tickets), { agente: 1, estado: 1, createdAt: -1 } (cola del agente), { estado: 1,

prioridad: -1, createdAt: -1 } (filtros del panel), { ticket: 1, createdAt: 1 } en comentarios y { email: 1 } único en usuarios. Se verificó con explain('executionStats'): IXSCAN en lugar de COLLSCAN.

- **Dónde se guarda el token y por qué.** En una **cookie HttpOnly, Secure y SameSite**: JavaScript no puede leerla, por lo que un ataque XSS no roba la sesión; SameSite reduce CSRF. localStorage es legible por cualquier script inyectado.
- **Qué pasa si dos agentes toman el mismo ticket a la vez.** Se evita con una **actualización atómica condicional**: Ticket.findOneAndUpdate({ _id, agente: null }, { agente: req.user.id, estado: 'en_progreso' }). Solo una de las dos operaciones encuentra el ticket sin agente; la otra recibe null y el servidor responde 409 ("ya fue tomado"). También sirve el control de versiones de Mongoose (optimisticConcurrency).
- **Qué fallaría si el frontend no re-renderiza tras actualizar un ticket.** Casi siempre se debe a **mutar el estado** (por ejemplo, t.estado = ... y setTickets(sameArray)) o a olvidar actualizar el estado tras la respuesta de la API. Se corrige con actualizaciones inmutables (map con objetos nuevos) o volviendo a pedir los datos (recargar del useFetch).
- **Cómo se protegió la API contra inyección NoSQL y abuso.** Validación de entrada con zod/joi (todo debe ser el tipo esperado), express-mongo-sanitize, lista blanca de campos al crear/editar (no usar req.body directo), helmet, rate limiting en /auth/login, bcrypt para contraseñas y mensajes de error genéricos.
- **Qué mejoraría con más tiempo.** Refresh tokens con rotación, paginación por cursor, pruebas e2e, caché de estadísticas, notificaciones por correo/WebSocket, adjuntos con almacenamiento externo y despliegue con CI/CD.

Rúbrica de evaluación (30 puntos)

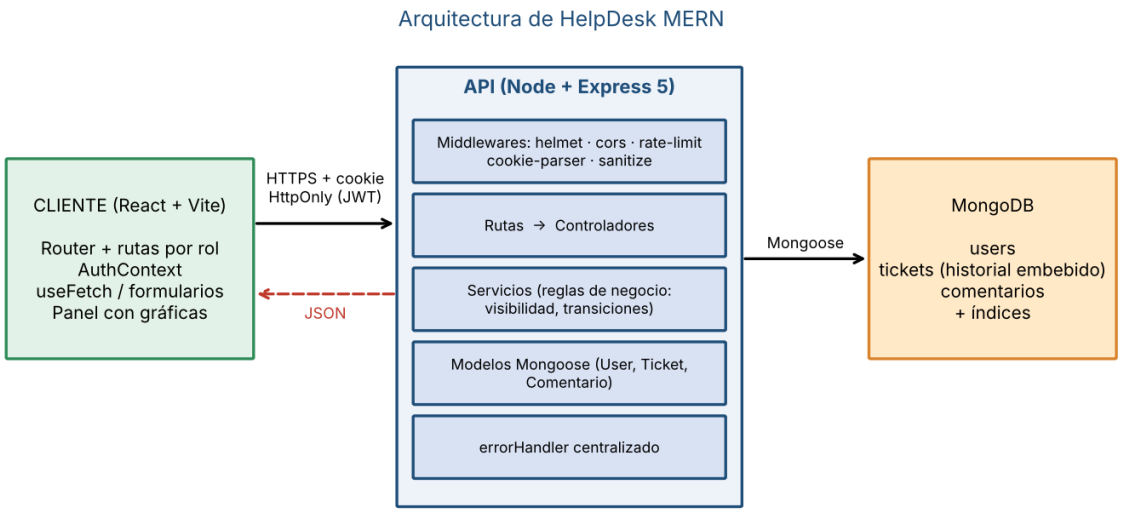
Criterio	Pts	Excelente (100 %)	Suficiente (60 %)	Insuficiente (0–30 %)
Modelado de datos e índices	4	Esquemas con validaciones; embebido y referencias justificados; índices alineados a las consultas y verificados con explain	Modelo funcional con algunas decisiones sin justificar o índices incompletos	Modelo inconsistente, sin validaciones ni índices
API REST con Express y arquitectura	5	Capas bien separadas, verbos y códigos HTTP correctos, errores centralizados y uniformes, validación de entrada en todas las rutas	API funcional con mezcla de responsabilidades o códigos inconsistentes	Lógica en las rutas, sin manejo de errores
Autenticación, autorización y seguridad	5	JWT en cookie HttpOnly, autorización por rol y por propiedad en el servidor, transiciones validadas, helmet, CORS, rate limit y protección contra inyección	Autenticación funcional con autorización solo en el frontend o con controles de seguridad faltantes	Contraseñas sin hash, token en localStorage sin justificar o rutas sin proteger
Frontend en React	5	Componentes reutilizables, hooks bien usados, rutas protegidas por rol, estado global con Context o similar, estados de carga, error y vacío, interfaz responsiva y accesible	Interfaz funcional con duplicación de código o estados de carga y error incompletos	Interfaz incompleta o con errores frecuentes

Criterio	Pts	Excelente (100 %)	Suficiente (60 %)	Insuficiente (0–30 %)
Integración y funcionalidad completa	4	Todos los RF operan de extremo a extremo para los tres roles	Faltan uno o dos RF o hay fallos menores	Faltan varios RF
Agregaciones y panel de métricas	2	Pipelines correctos (por ejemplo, con $facet y $lookup) que respetan la visibilidad del rol; gráficas claras	Métricas parciales o calculadas en JavaScript en lugar de en la base de datos	Sin panel
Pruebas	3	8 o más pruebas de API y 3 o más de componentes, significativas (incluyen casos de autorización y error) y aisladas	Menos pruebas de las requeridas o solo casos exitosos	Sin pruebas
Documentación, Git y entrega	2	README completo, seed, colección de la API, video y commits descriptivos	Documentación parcial	Sin README o sin instrucciones para ejecutar

Propuesta de solución – Parte III: HelpDesk MERN

Alcance de esta sección. El proyecto completo es un entregable de dos semanas (repositorio, pruebas, despliegue, demostración), por lo que no cabe en un documento de examen. Aquí se incluye una **solución de referencia**: arquitectura, modelo de datos, índices, seguridad, las reglas de negocio con su código clave, estadísticas, pruebas y plan de trabajo, de manera que se pueda construir el proyecto siguiendo este guion.

1) Arquitectura



2) Estructura de carpetas sugerida

```
helpdesk/
├─ server/
│  ├─ src/
│  │  ├─ config/        (env.js, db.js)
│  │  ├─ models/        (User.js, Ticket.js, Comentario.js)
│  │  ├─ middlewares/   (auth.js, validar.js, errorHandler.js)
│  │  ├─ services/      (ticketService.js, statsService.js)
│  │  ├─ controllers/   (authController.js, ticketController.js, ...)
│  │  ├─ routes/        (auth.js, tickets.js, usuarios.js, stats.js)
│  │  ├─ validators/    (esquemas zod)
│  │  └─ app.js · server.js
│  ├─ tests/            (auth.test.js, tickets.test.js)
│  ├─ scripts/seed.js
│  └─ .env.example
└─ client/
   ├─ src/
   │  ├─ context/AuthContext.jsx
   │  ├─ hooks/useFetch.js
   │  ├─ components/    (RutaProtegida, TicketCard, Filtros, FormularioTicket...)
   │  └─ pages/         (Login, MisTickets, DetalleTicket, ColaAgente, Panel)
   └─ vite.config.js    (proxy /api → http://localhost:3000)
```

3) Modelo de datos

Colección	Campos principales	Decisión de diseño
users	nombre, email (único), password (hash, select:false), rol, activo, timestamps	Una colección para los tres roles; el rol viaja en el token.

Colección	Campos principales	Decisión de diseño
tickets	titulo, descripcion, categoria, prioridad, estado, cliente (ref), agente (ref, opcional), historial[] (embebido: estado, por, fecha, nota), timestamps	El **historial se embebe** (acotado; se lee con el ticket y se actualiza atómicamente).
comentarios	ticket (ref), autor (ref), texto, interno (bool), createdAt	**Colección aparte**: crecimiento sin límite y paginación. interno = nota visible solo para agentes/admin.

```
// models/Ticket.js (resumen)
const ESTADOS = ['abierto', 'en_progreso', 'resuelto', 'cerrado'];

const ticketSchema = new mongoose.Schema({
  titulo:      { type: String, required: true, trim: true, maxlength: 120 },
  descripcion: { type: String, required: true, trim: true, maxlength: 4000 },
  categoria:   { type: String, enum: ['hardware', 'software', 'red', 'otro'], required: true
},
  prioridad:   { type: String, enum: ['baja', 'media', 'alta', 'critica'], default: 'media' },
  estado:      { type: String, enum: ESTADOS, default: 'abierto' },
  cliente:     { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  agente:      { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null },
  historial: [{
    estado: { type: String, enum: ESTADOS },
    por:    { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    nota:   String,
    fecha:  { type: Date, default: Date.now },
    _id: false,
  }],
}, { timestamps: true, optimisticConcurrency: true });

ticketSchema.index({ cliente: 1, createdAt: -1 });
ticketSchema.index({ agente: 1, estado: 1, createdAt: -1 });
ticketSchema.index({ estado: 1, prioridad: -1, createdAt: -1 });
```

4) Índices y consultas que aceleran

Índice	Consulta que acelera
users: { email: 1 } único	Login y evitar correos duplicados.
tickets: { cliente: 1, createdAt: -1 }	“Mis tickets” del cliente, del más reciente al más antiguo.
tickets: { agente: 1, estado: 1, createdAt: -1 }	Cola de trabajo de un agente filtrada por estado.
tickets: { estado: 1, prioridad: -1, createdAt: -1 }	Panel y filtros del administrador; tickets sin asignar (agente: null).
comentarios: { ticket: 1, createdAt: 1 }	Comentarios de un ticket, paginados y en orden.

5) Autenticación y roles (RF de seguridad)

- POST /api/auth/register (rol cliente por defecto; **el rol nunca se toma del body**), POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me.
- Contraseñas con bcrypt (coste 12), JWT en cookie **HttpOnly + Secure + SameSite**, expiración corta (ver respuesta 64), rate limit en login, helmet, cors con origen explícito y credenciales, express-mongo-sanitize y validación con zod. Los middlewares autenticar y autorizar son los de P4.

6) Visibilidad por rol y transiciones de estado (servicio)

```
// services/ticketService.js
export function filtroVisibilidad(user) {
  if (user.rol === 'admin') return {};
  if (user.rol === 'agente') return { $or: [{ agente: user.id }, { agente: null }] };
  return { cliente: user.id };                        // cliente: solo los suyos
}

export async function obtenerTicket(user, id) {
  if (!mongoose.isValidObjectId(id)) throw httpError(400, 'Id inválido');
  const t = await Ticket.findOne({ _id: id, ...filtroVisibilidad(user) });
  if (!t) throw httpError(404, 'Ticket no encontrado');   // 404: no revela que existe
  return t;
}

// Transiciones válidas y quién puede hacerlas
const TRANSICIONES = {
  abierto:     { en_progreso: ['agente', 'admin'] },
  en_progreso: { resuelto: ['agente', 'admin'], abierto: ['admin'] },
  resuelto:    { cerrado: ['cliente', 'admin'], en_progreso: ['cliente', 'agente'] }, //
reapertura
  cerrado:     {},
};

export async function cambiarEstado(user, id, nuevo, nota) {
  const t = await obtenerTicket(user, id);
  const roles = TRANSICIONES[t.estado]?.[nuevo];
  if (!roles) throw httpError(409, `Transición no permitida: ${t.estado} → ${nuevo}`);
  if (!roles.includes(user.rol)) throw httpError(403, 'Su rol no puede hacer este cambio');
  t.estado = nuevo;
  t.historial.push({ estado: nuevo, por: user.id, nota });
  return t.save();            // optimisticConcurrency detecta ediciones simultáneas
}

// Tomar un ticket: atómico, evita que dos agentes lo tomen a la vez
export async function tomarTicket(user, id) {
  const t = await Ticket.findOneAndUpdate(
    { _id: id, agente: null, estado: 'abierto' },
    { agente: user.id, estado: 'en_progreso',
      $push: { historial: { estado: 'en_progreso', por: user.id, nota: 'Tomado' } } },
    { new: true });
  if (!t) throw httpError(409, 'El ticket ya fue tomado o no está disponible');
  return t;
}
```

7) Endpoints mínimos

Método y ruta	Quién	Resumen
POST /api/tickets	cliente	Crea ticket (estado abierto, cliente = usuario del token).
GET /api/tickets?estado=&prioridad=&page=&limit=	todos	Lista filtrada y paginada según visibilidad por rol.
GET /api/tickets/:id	todos (con permiso)	Detalle + historial; comentarios paginados aparte.
PATCH /api/tickets/:id/estado	según transición	Valida la transición y agrega al historial.
POST /api/tickets/:id/tomar	agente	Asignación atómica.

Método y ruta	Quién	Resumen
PATCH /api/tickets/:id/asignar	admin	Asigna o reasigna a un agente.
POST /api/tickets/:id/comentarios	participantes	Agrega comentario (interno solo agente/admin).
GET /api/stats/resumen	admin	Estadísticas (ver abajo).
GET/POST/PATCH /api/usuarios	admin	Alta de agentes, cambio de rol y desactivación.

8) Estadísticas del panel (una sola consulta con $facet)

```
// services/statsService.js
export function resumen() {
  return Ticket.aggregate([
    { $facet: {
        porEstado:    [{ $group: { _id: '$estado', total: { $sum: 1 } } }],
        porPrioridad: [{ $group: { _id: '$prioridad', total: { $sum: 1 } } }],
        porAgente: [
          { $match: { agente: { $ne: null } } },
          { $group: { _id: '$agente', total: { $sum: 1 } } },
          { $lookup: { from: 'users', localField: '_id', foreignField: '_id', as: 'agente' }
},
          { $unwind: '$agente' },
          { $project: { _id: 0, nombre: '$agente.nombre', total: 1 } },
        ],
        resolucionPromedioHoras: [
          { $match: { estado: { $in: ['resuelto', 'cerrado'] } } },
          { $group: { _id: null,
              horas: { $avg: { $divide: [{ $subtract: ['$updatedAt', '$createdAt'] }, 3600000]
} } } },
        ],
    } },
  ]);
}
```

9) Frontend (React + Vite)

- **AuthContext** guarda el usuario (se carga con GET /api/auth/me al iniciar) y expone login/logout.
- **Rutas protegidas por rol** con React Router:

```
function RutaProtegida({ roles, children }) {
  const { user, cargando } = useContext(AuthContext);
  if (cargando) return <p>Cargando…</p>;
  if (!user) return <Navigate to="/login" replace />;
  if (roles && !roles.includes(user.rol)) return <Navigate to="/" replace />;
  return children;
}

<Route path="/panel" element={<RutaProtegida roles={['admin']}><Panel /></RutaProtegida>} />
```

- Vistas: Login/Registro, **Mis tickets** (cliente) con filtros y formulario de creación, **Detalle** (historial, comentarios y botones de cambio de estado según rol), **Cola del agente** y **Panel del administrador** con gráficas (recharts) de las estadísticas.
- Datos con el hook useFetch (P8); formularios controlados; mensajes de error/éxito; estados de carga y de lista vacía.

10) Pruebas mínimas (Vitest + Supertest + mongodb-memory-server)

```
describe('Tickets', () => {
  it('un cliente no puede ver tickets ajenos (404)', async () => {
    const res = await agenteA.get(`/api/tickets/${ticketDeOtroCliente.id}`);
    expect(res.status).toBe(404);
  });

  it('rechaza una transición inválida (abierto → cerrado) con 409', async () => {
    const res = await admin.patch(`/api/tickets/${t.id}/estado`).send({ estado: 'cerrado' });
    expect(res.status).toBe(409);
  });

  it('solo uno de dos agentes puede tomar el mismo ticket', async () => {
    const [r1, r2] = await Promise.all([
      agente1.post(`/api/tickets/${t.id}/tomar`),
agente2.post(`/api/tickets/${t.id}/tomar`)]);
    expect([r1.status, r2.status].sort()).toEqual([200, 409]);
  });

  it('responde 401 sin cookie de sesión', async () => {
    expect((await request(app).get('/api/tickets')).status).toBe(401);
  });
});
```

11) Semilla de datos, variables de entorno y entrega

- **scripts/seed.js** crea: 1 admin, 2 agentes, 3 clientes y ≥ 20 tickets en todos los estados con historial y comentarios (npm run seed).

```
# .env.example
PORT=3000
MONGODB_URI=mongodb://localhost:27017/helpdesk
JWT_SECRET=cambia_esto_por_un_valor_largo_y_aleatorio
CLIENT_ORIGIN=http://localhost:5173
NODE_ENV=development
```

- **README** con instalación, variables de entorno, scripts, usuarios de prueba y decisiones de diseño; .gitignore con node_modules y .env; commits pequeños y descriptivos.

12) Plan de trabajo sugerido (2 semanas)

Día	Entregable
1–2	Repositorio, configuración, conexión a MongoDB, modelos y semilla.
3–4	Autenticación (registro, login, cookie, middlewares) y gestión de usuarios.
5–6	CRUD de tickets con visibilidad por rol, validaciones y comentarios.
7	Transiciones de estado, historial y asignación atómica.
8–9	Estadísticas ($facet), índices y verificación con explain().
10–11	Frontend: AuthContext, rutas protegidas, vistas de cliente y agente.
12	Panel con gráficas, manejo de errores y estados de carga.
13	Pruebas, revisión de seguridad (helmet, rate limit, sanitización) y README.
14	Ensayo de la demo y de la defensa oral.